Deusto
Facultad de Ingeniería
Ingeniaritza Fakultatea

Grado en Ingeniería Informática

Informatikako Ingeniaritzako gradua

Proyecto fin de grado

Gradu amaierako proiektua

Titulo de prueba

Juan Nadie

Director/a: John Doe

Bilbao, junio de 2023 (mes de convocatoria de evaluación)

Deusto
Facultad de Ingeniería
Ingeniaritza Fakultatea

Grado en Ingeniería Informática

Informatikako Ingeniaritzako gradua

Proyecto fin de grado

Gradu amaierako proiektua

Titulo de prueba

Juan Nadie

Director/a: John Doe

Bilbao, junio de 2023 (mes de convocatoria de evaluación)

Abstract

This work presents CALA-SafeRL, a modular framework for Safe Reinforcement Learning applied to autonomous driving in the CARLA simulator. The central challenge addressed is how to train a Proximal Policy Optimization (PPO) agent that learns to drive under realistic traffic conditions without catastrophic failures during the learning process itself. The framework is structured as a composable chain of Gymnasium wrappers, allowing the agent, two interchangeable safety shields, a dense reward shaper and the simulator to be combined with minimal coupling. The first shield performs a lightweight threshold-based verification on semantic LIDAR readings together with CARLA's Waypoint API; the second implements an adaptive-horizon strategy, predicting the vehicle's future trajectory with a kinematic bicycle model calibrated for a Tesla Model 3 and selecting the least invasive corrective action from a prioritised candidate set. The agent's observation space has 735 dimensions, combining three semantic LIDAR channels with lane, vehicle and route features derived from the Waypoint API. Training stability is improved through running-mean observation normalisation, return scaling, Generalised Advantage Estimation, clipped surrogate updates with a target-KL early stop, and a performance-driven curriculum that progressively introduces traffic density. The framework is accompanied by a live metrics pipeline built on SQLite and a Streamlit dashboard. Experiments are structured around the Town04 highway map and systematically compare the three shielding configurations against baseline PPO. The resulting system is open, reproducible and can serve as a foundation for further research into safety-aware policy learning.

Contents

List of Figures

List of Tables

Índice de Seudocódigos

1. INTRODUCTION

1.1. CONTEXT AND MOTIVATION

Road transport is one of the dominant causes of avoidable injury and death worldwide. According to the World Health Organization, approximately 1.19 million people die every year as a direct consequence of road traffic crashes, with human error being the leading contributing factor [1]. This figure has motivated a sustained effort, both in industry and in academia, to automate vehicle control up to the point where the driver can be completely displaced from the control loop. The SAE J3016 standard formalises this progression in six levels, from no automation (Level 0) to full self-driving with no operational design domain restrictions (Level 5) [2]. While most commercially deployed systems remain at Level 2, research platforms increasingly target Levels 4 and 5, where the vehicle must cope with the full variety of traffic situations, weather conditions and road topologies encountered in real-world operation.

Among the techniques that have received growing attention for high-autonomy driving, Reinforcement Learning (RL) [3] occupies a distinctive position. Classical model-predictive and rule-based controllers require a hand-crafted description of every situation and a correspondingly explicit control law; in contrast, an RL agent learns a policy end-to-end by interacting with an environment and maximising a scalar reward signal. Recent surveys of deep RL for autonomous driving report promising results on lane keeping, overtaking, intersection handling and dense urban scenarios [4, 5].

The main obstacle to deploying these systems, however, is not their peak performance but their behaviour *during learning*. Vanilla RL relies on stochastic exploration, which is incompatible with the safety requirements of a vehicle operating on public roads: even a single exploratory collision is unacceptable. The research field generally referred to as *Safe Reinforcement Learning* (Safe RL) addresses this tension through multiple strategies [6]: encoding constraints in the reward function, constraining the policy optimisation problem itself, or intercepting unsafe actions with a runtime verifier known as a *safety shield* [7]. This project belongs to the latter family: a PPO agent [8] is trained with two alternative shields that monitor its proposed actions, reason about their near-future consequences, and substitute them when the resulting trajectory is judged unsafe. The shield layer runs entirely during training, so safety violations are not merely discouraged by the reward function but actively prevented at the actuator.

1.2. PROBLEM STATEMENT

The central problem addressed in this thesis can be stated as follows. A vehicle, represented as an RL agent, drives inside the CARLA simulator [9]. The agent observes a partial representation of the environment through virtual sensors and a limited number of kinematic variables, and must produce low-level control commands (steering, throttle and brake). The goal is to learn a policy that

1. *maximises* a task reward that encodes progress, speed compliance and lane keeping,
2. *minimises* unsafe events such as collisions, off-road excursions and stalls, and
3. *tolerates* the shift in traffic density that occurs as the scenario is progressively complicated during training.

The research question that guides the work is then: *can a lightweight, interchangeable safety-shield layer, combined with a dense reward shaper and a performance-driven curriculum, train a PPO agent that is competitive with unconstrained PPO in terms of reward while keeping crash and off-road rates close to zero during learning?*

1.3. OBJECTIVES

General objective

To design, implement and empirically analyse a modular Safe RL framework for autonomous driving on the CARLA simulator, combining a PPO agent with interchangeable safety shields, a dense reward shaper and a curriculum manager.

Specific objectives

SO1. Build a Gymnasium-compatible wrapper of the CARLA Python API that exposes a 735-dimensional observation space (three semantic LIDAR channels plus lane, vehicle and route features) and a continuous two-dimensional action space, running in synchronous mode at 20 Hz.

SO2. Implement a PPO agent with generalised advantage estimation, clipped surrogate updates, value clipping, a target-KL early stop, cosine learning-rate annealing and online normalisation of the vector observation block.

SO3. Develop a basic safety shield that combines LIDAR thresholds with information from CARLA's Waypoint

API and applies a proportional correction to steering and brake whenever the agent's action is deemed unsafe.

SO4. Develop an adaptive-horizon safety shield that classifies the current situation into three risk levels, predicts the vehicle's trajectory with a kinematic bicycle model, and selects the least invasive corrective action from a priority-ordered candidate set.

SO5. Design a dense reward shaper that combines the sparse CARLA rewards with components for velocity tracking, lane centring, heading alignment, smoothness, progress, idle avoidance and lane-change tolerance.

SO6. Build a performance-driven curriculum that progressively introduces non-playable vehicles (NPCs) into the scenario, with automatic rollback to earlier stages on performance collapse.

SO7. Provide a live metrics pipeline based on SQLite (WAL mode) and a Streamlit dashboard so that each training run can be inspected while it is still in progress.

1.4. SCOPE AND LIMITATIONS

The scope of this project is deliberately bounded to keep the contribution verifiable and reproducible within the duration of a Bachelor's thesis.

- The simulator version is *CARLA 0.9.16*. Compatibility with earlier or future versions is not guaranteed.
- The main benchmark map is *Town04*, a highway-oriented layout that is well suited for lane-keeping and overtaking experiments. Urban maps (Town01–Town03) and the maximum difficulty map (Town05) are supported by the framework but not systematically evaluated.
- Two shielding strategies are developed and compared: *basic* (threshold + Waypoint API correction) and *adaptive-horizon* (trajectory prediction with a kinematic bicycle model). Alternatives based on reachability analysis, model-predictive safety filters or formal verification are left for future work.
- The vehicle model used by the adaptive shield is the kinematic bicycle model [10]. A dynamic model with tyre slip is not considered; at the speeds and lateral accelerations encountered in the experiments the kinematic approximation is a reasonable trade-off between fidelity and computational cost.
- Perception is based on semantic LIDAR plus CARLA's Waypoint API. Camera-based perception, sensor

fusion and learned occupancy grids are not part of the observation space.

- The framework trains a *single ego vehicle*. Multi-agent safe RL is an explicit direction for future extensions.

1.5. METHODOLOGY OVERVIEW

The project follows an iterative engineering methodology structured around four phases: design, implementation, training and analysis, which are revisited in successive loops as new empirical evidence becomes available.

1. **Design.** The software architecture is expressed as a layered wrapper chain on top of Gymnasium. Each layer (environment, shield, reward shaper) has a single responsibility and a well-defined interface, so that components can be swapped or reconfigured without touching the others.

2. **Implementation.** The Python code base is organised into self-contained modules (`src/CARLA/Env`, `src/PPO`, `src/Adaptative_Shield`, `src/reward_shaper.py`, `src/curriculumManager.py`, `src/Metrics`). Automated style checks with Ruff and targeted unit tests on the reward shaper prevent regressions.

3. **Training.** Each experimental configuration is launched via `main_train.py` with explicit command-line flags. The agent writes metrics to a per-run SQLite database and checkpoints every 200 episodes.

4. **Analysis.** The `dataVisualizer.py` Streamlit dashboard is used to inspect reward curves, safety rates, curriculum transitions and shield activations during and after training. Observed pathologies are translated into new design iterations.

1.6. DOCUMENT STRUCTURE

The remainder of this document is organised as follows.

Chapter 2 reviews the theoretical background. It introduces Markov Decision Processes, derives the PPO objective from the policy gradient, describes the Generalised Advantage Estimator, surveys the main families of Safe RL, formalises the kinematic bicycle model and summarises the features of the CARLA simulator that are relevant to this project.

Chapter 3 captures the functional and non-functional requirements that the framework must satisfy, together

with the technical constraints imposed by the simulator, the hardware and the Bachelor-thesis timeline.

Chapter 4 describes the system design: the wrapper chain, the structure of the observation and action spaces, the Actor–Critic network, the two safety shields, the reward shaper and the curriculum manager.

Chapter 5 details the implementation: technology stack, project layout, key algorithms in pseudocode, training and evaluation pipelines, and reproducibility considerations.

Chapter 6 presents the experimental protocol and a structured template to report the empirical results obtained on Town04.

Chapter ?? summarises the contributions, evaluates the fulfilment of the objectives enumerated above, and proposes future work.

Appendices A, B and C provide the complete list of default hyperparameters, a full breakdown of the observation space and a reproducibility guide, respectively.

2. STATE OF THE ART AND THEORETICAL FRAMEWORK

This chapter reviews the concepts that underpin the rest of the document. It first formalises the Reinforcement Learning problem (Section 2.1), then derives Proximal Policy Optimization and Generalised Advantage Estimation (Sections 2.2–2.3), surveys the Safe RL literature with a focus on shielding (Sections 2.4–2.5), introduces the kinematic bicycle model used by the adaptive shield (Section 2.6), and finishes with an overview of the CARLA simulator and related work (Sections 2.7–2.8).

2.1. REINFORCEMENT LEARNING FUNDAMENTALS

The standard formulation of Reinforcement Learning relies on Markov Decision Processes (MDPs) [3, 11]. A (discrete-time, discounted) MDP is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0)$ where:

- $\mathcal{S}$ is the set of states,
- $\mathcal{A}$ is the set of actions,
- $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the transition kernel with $P(s' \mid s, a)$ denoting the probability of reaching s' from s after taking a,
- $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function,
- $\gamma \in [0, 1)$ is the discount factor, and
- ρ_0 is the initial-state distribution.

A stochastic policy $\pi_\theta : \mathcal{S} \to \Delta(\mathcal{A})$, parameterised by $\theta \in \mathbb{R}^d$, induces a distribution over trajectories $\tau = (s_0, a_0, s_1, a_1, \ldots)$. The *return* of a trajectory is the discounted sum of rewards,

$$G(\tau) \;=\; \sum_{t=0}^{\infty} \gamma^t \, r(s_t, a_t), \tag{2.1}$$

and the objective of the agent is to maximise the expected return:

$$J(\theta) \;=\; \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)] \,. \tag{2.2}$$

Two central quantities are the state-value function and the action-value function, which describe the expected return from a given state (respectively state–action pair) when the policy π_θ is followed thereafter:

$$V^{\pi_\theta}(s) \;=\; \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t\, r(s_t, a_t) \,\middle|\, s_0 = s\right], \tag{2.3}$$

$$Q^{\pi_\theta}(s, a) \;=\; \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t\, r(s_t, a_t) \,\middle|\, s_0 = s,\; a_0 = a\right]. \tag{2.4}$$

The *advantage* of an action is defined as $A^{\pi_\theta}(s, a) = Q^{\pi_\theta}(s, a) - V^{\pi_\theta}(s)$ and measures how much better (or worse) the action is than average at state s.

2.1.1. Policy gradient theorem

Rather than learning a value function explicitly and deriving the policy from it (as in Q-learning), policy gradient methods directly optimise Equation (2.2) by gradient ascent on θ. The policy gradient theorem [12] provides a tractable expression of $\nabla_\theta J(\theta)$:

$$\nabla_\theta J(\theta) \;=\; \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\; A^{\pi_\theta}(s_t, a_t)\right]. \tag{2.5}$$

Equation (2.5) is the cornerstone of modern on-policy methods such as A2C/A3C, TRPO [13] and PPO [8]. In practice, the advantage $A^{\pi_\theta}(s_t, a_t)$ is replaced by a sample-based estimator $\hat{A}_t$; the quality of this estimator has a direct impact on the variance of the gradient and, through it, on the sample efficiency of the algorithm.

2.2. PROXIMAL POLICY OPTIMIZATION

Vanilla policy gradient is notoriously sensitive to the step size: a single large update can move the policy so far away from the current one that the next rollout is already off-distribution. TRPO [13] addresses this by constraining the Kullback–Leibler divergence between the old and the new policy at each update:

$$\max_\theta \; \mathbb{E}\left[\frac{\pi_\theta(a \mid s)}{\pi_{\theta_{\text{old}}}(a \mid s)}\, \hat{A}(s, a)\right] \quad \text{s.t.} \quad \mathbb{E}[D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot \mid s) \,\|\, \pi_\theta(\cdot \mid s))] \le \delta. \tag{2.6}$$

The trust-region constraint in Equation (2.6) is expensive to enforce because it requires second-order information. PPO [8] replaces it with a far cheaper first-order surrogate. Let

$$r_t(\theta) \;=\; \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \tag{2.7}$$

denote the importance-sampling ratio. The *clipped surrogate objective* of PPO is

$$\mathcal{L}^{\mathrm{CLIP}}(\theta) \;=\; \mathbb{E}_t\Big[\min\Big(r_t(\theta)\,\hat{A}_t,\; \mathrm{clip}\big(r_t(\theta),\,1-\epsilon,\,1+\epsilon\big)\,\hat{A}_t\Big)\Big], \tag{2.8}$$

where ϵ is a small hyperparameter (typically $\epsilon = 0.2$). The clipping term prevents the ratio $r_t(\theta)$ from drifting too far from one, thereby keeping the update close to a trust region without ever computing a Hessian. The full loss minimised by PPO also includes a value-function error and an entropy bonus:

$$\mathcal{L}^{\mathrm{PPO}}(\theta) \;=\; -\,\mathcal{L}^{\mathrm{CLIP}}(\theta) \;+\; c_{\mathrm{V}}\,\mathcal{L}^{\mathrm{V}}(\theta) \;-\; c_{\mathrm{H}}\,\mathcal{H}[\pi_\theta], \tag{2.9}$$

with

$$\mathcal{L}^{\mathrm{V}}(\theta) \;=\; \mathbb{E}_t\Big[\big(V_\theta(s_t) - \hat{V}_t^{\text{target}}\big)^2\Big], \qquad \mathcal{H}[\pi_\theta] \;=\; \mathbb{E}_t\Big[-\sum_a \pi_\theta(a \mid s_t)\log\pi_\theta(a \mid s_t)\Big]. \tag{2.10}$$

The coefficients c_{V} and c_{H} balance the three terms. The entropy bonus encourages exploration, while the value loss regresses the critic towards a bootstrapped target $\hat{V}_t^{\text{target}}$ that, in this work, is computed from the GAE estimator of Section 2.3.

2.2.1. Stabilisers and heuristics

Empirical studies [14, 15] have shown that a number of implementation-level details can have a first-order effect on PPO's sample efficiency and stability. The ones that are relevant to this project are listed below; each of them corresponds to a configurable component in Chapter 4.

Value clipping. The value head is updated by minimising

$$\mathcal{L}^{\mathrm{V,clip}}(\theta) \;=\; \mathbb{E}_t\Big[\max\Big(\big(V_\theta(s_t) - \hat{V}_t^{\text{target}}\big)^2,\; \big(\mathrm{clip}(V_\theta(s_t),\, V_{\theta_{\text{old}}}(s_t)-\xi,\, V_{\theta_{\text{old}}}(s_t)+\xi) - \hat{V}_t^{\text{target}}\big)^2\Big)\Big], \tag{2.11}$$

which prevents the critic from making overly aggressive updates inside a single optimisation batch.

KL early stopping. After every minibatch pass the approximate KL divergence $\widehat{D}_{\mathrm{KL}}$ between the new and the old policy is computed via the unbiased estimator of [16]:

$$\widehat{D}_{\mathrm{KL}} \;=\; \mathbb{E}_t\big[\big(e^{\ell_t} - 1\big) - \ell_t\big], \qquad \ell_t = \log\pi_\theta(a_t \mid s_t) - \log\pi_{\theta_{\text{old}}}(a_t \mid s_t). \tag{2.12}$$

If $\widehat{D}_{\mathrm{KL}} > D^{\star}_{\mathrm{KL}}$ (target) the remaining epochs of the current update are skipped. This mechanism guards the policy against destructive updates when a minibatch happens to contain a particularly informative

trajectory.

Observation normalisation. Each component of the state s_t is rescaled to zero mean and unit variance using a running estimator. In this project only the 15-dimensional vector block is normalised, the LIDAR block is not; the reason is discussed in Section 4.4.3 of Chapter 4.

Cosine learning-rate annealing. The learning rate decays smoothly from $\eta_{\max}$ to $\eta_{\min}$ following

$$\eta_k \;=\; \eta_{\min} \;+\; \tfrac{1}{2}\,(\eta_{\max} - \eta_{\min})\,\Big(1 + \cos\big(\pi\, k / K_{\max}\big)\Big), \tag{2.13}$$

where k is the current PPO update and $K_{\max}$ is the number of expected updates over the entire training run.

2.3. GENERALISED ADVANTAGE ESTIMATION

The advantage $A^{\pi_\theta}(s_t, a_t)$ appearing in Equation (2.5) has to be estimated from sampled rollouts. There is a bias-variance trade-off between short-horizon estimators (low variance, high bias) and Monte Carlo returns (zero bias, high variance). Generalised Advantage Estimation (GAE) [17] interpolates between the two via an exponentially-weighted sum of *temporal-difference* residuals.

Let

$$\delta_t^V \;=\; r_t + \gamma\, V_\theta(s_{t+1}) - V_\theta(s_t) \tag{2.14}$$

be the one-step TD residual. GAE defines the advantage as

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} \;=\; \sum_{\ell=0}^{\infty} (\gamma\lambda)^\ell\, \delta_{t+\ell}^V, \tag{2.15}$$

where $\lambda \in [0, 1]$ is the GAE parameter. Two extreme cases are useful to bear in mind:

- $\lambda = 0$ yields $\hat{A}_t = \delta_t^V$, i.e. the one-step TD residual.
- $\lambda = 1$ yields the (discounted) Monte Carlo advantage.

Intermediate values (in this work $\lambda \;=\; 0.95$) provide a practical balance. When a trajectory is truncated at step T because a timeout is reached, the missing tail of Equation (2.15) is bootstrapped with $V_\theta(s_{T+1})$; when the episode terminates naturally (collision, off-road), $V_\theta(s_{T+1}) = 0$ is used instead.

The target for the value loss in Equation (2.10) is the advantage plus the current value estimate:

$$\hat{V}_t^{\text{target}} = \hat{A}_t^{\text{GAE}(\gamma,\lambda)} + V_\theta(s_t). \tag{2.16}$$

2.4. SAFE REINFORCEMENT LEARNING

The comprehensive survey by García and Fernández [6] organises Safe RL methods into two broad categories: *risk-sensitive* approaches that modify the optimality criterion, and *exploration-modifying* approaches that constrain the set of actions available to the agent during learning. Within these two categories, the following four families are particularly relevant to autonomous driving.

- **Constrained MDPs and CPO.** Safety is expressed as one or more auxiliary cost functions $c_i(s, a)$, and the policy is trained to maximise reward subject to $\mathbb{E}_\tau[\sum_t c_i(s_t, a_t)] \leq d_i$. Achiam et al. [18] introduced *Constrained Policy Optimization* (CPO), a TRPO-style algorithm that guarantees approximately monotonic constraint satisfaction.

- **Lagrangian and primal-dual methods.** A more scalable alternative to CPO is to form the Lagrangian $\mathcal{L}(\theta, \mu) = J(\theta) - \mu(C(\theta) - d)$ and solve it with alternating updates on θ and μ. Variants of this idea underpin a large body of modern Safe RL work.

- **Safety-aware reward shaping.** Here the reward is augmented with penalty terms that encourage the agent to avoid undesirable states. This is a lightweight form of safety supervision but does not provide hard guarantees: the agent can still try unsafe actions during exploration. The reward shaper described in Section 4.6 of Chapter 4 belongs to this family.

- **Runtime shielding.** A verifier, typically based on formal methods or a predictive model of the system, inspects each action the agent proposes and substitutes it by a fall-back action if a safety specification would be violated. Shielding is orthogonal to the training algorithm and can be composed with any RL backbone. Alshiekh et al. [7] formalise the approach for finite systems with temporal-logic specifications, and Jansen et al. [19] extend it to probabilistic settings. The two shields implemented in this project are runtime shields of the second type: they reason about the vehicle's predicted trajectory rather than using formal verification.

2.5. SAFETY SHIELDS

A runtime safety shield can be formalised as a function

$$\Sigma : \mathcal{S} \times \mathcal{A} \to \mathcal{A}, \qquad a_t^\star \;=\; \Sigma(s_t,\, a_t), \tag{2.17}$$

that, given the current state s_t and the action a_t proposed by the agent, returns an action $a_t^\star$ that is guaranteed to satisfy a pre-specified safety property. Two design principles recur throughout the literature:

Minimal interference. The shield should override the agent's action only when strictly necessary, and the override should be as close as possible to the proposed action. In practice this is implemented either by projecting the agent's action onto a set of certified-safe actions, or by searching through an ordered list of safe candidates.

Credit assignment through the executed action. When the shield does intervene, the action that actually reaches the environment is $a_t^\star$, not a_t. If the replay buffer stores the proposed action, the RL update will push the policy towards the action that was never executed, which is incorrect. The framework presented here addresses this by storing the executed action and re-computing its log probability under the current policy, as discussed in Section 5.4.1.

Shields are frequently classified as *pre-posed* or *post-posed*. A pre-posed shield restricts the set of admissible actions *before* the agent samples from its policy, whereas a post-posed shield inspects the action produced by the policy and rewrites it. In this work, both shields are post-posed: they are inserted between the agent and the simulator as a `gymnasium.Wrapper`.

2.6. KINEMATIC BICYCLE MODEL

Analysing the safety of the trajectory induced by a candidate action requires a forward model of the vehicle. The adaptive shield developed in this thesis uses a *kinematic bicycle model*, a long-standing approximation in vehicle dynamics [10, 20] that captures the essential behaviour of a four-wheeled vehicle by collapsing each axle into a single virtual wheel located at its midpoint.

Let (x, y, θ) be the position and heading of the rear axle of the vehicle in a global Cartesian frame, v its longitudinal speed, δ the front-wheel steering angle and L the distance between the two axles (wheelbase).

The continuous-time equations of motion are

$$\dot{x} = v\cos\theta, \tag{2.18}$$

$$\dot{y} = v\sin\theta, \tag{2.19}$$

$$\dot{\theta} = \frac{v}{L}\tan\delta. \tag{2.20}$$

Equation (2.20) shows that the turning rate is proportional to the vehicle's speed and to the tangent of the steering angle, and inversely proportional to the wheelbase.

Integrating Equations (2.18)–(2.20) forward one time-step Δt with a forward-Euler scheme yields the discrete form used by the shield:

$$\theta_{t+1} = \theta_t + \frac{v_t}{L}\tan\delta_t \cdot \Delta t, \tag{2.21}$$

$$x_{t+1} = x_t + v_t\cos\theta_t \cdot \Delta t, \tag{2.22}$$

$$y_{t+1} = y_t + v_t\sin\theta_t \cdot \Delta t, \tag{2.23}$$

$$v_{t+1} = \max\big(0,\ v_t + a_t\,\Delta t\big), \tag{2.24}$$

where a_t is the longitudinal acceleration demanded by the throttle–brake action. In this project the throttle–brake command is mapped linearly to two saturation values: a positive acceleration of up to $3.0\,\mathrm{m/s^2}$ for throttle and a negative acceleration of up to $7.0\,\mathrm{m/s^2}$ for brake. A positive steering command is mapped linearly to a physical range $\delta \in [-\delta_{\max}, +\delta_{\max}]$ with $\delta_{\max} = 0.6\,\mathrm{rad}$.

When $|\delta_t|$ is below a small numerical threshold the vehicle is considered to be moving in a straight line and Equations (2.22)–(2.23) are applied directly. Otherwise, the trajectory traces an arc of radius $R = L/\tan|\delta_t|$ and the position update in Cartesian coordinates is expressed as

$$\theta_{t+1} = \theta_t + \frac{v_t}{|R|}\,\mathrm{sgn}(\delta_t)\,\Delta t, \tag{2.25}$$

$$x_{t+1} = x_t + |R|\,\mathrm{sgn}(R)\,\big(\sin\theta_{t+1} - \sin\theta_t\big), \tag{2.26}$$

$$y_{t+1} = y_t + |R|\,\mathrm{sgn}(R)\,\big(\cos\theta_t - \cos\theta_{t+1}\big). \tag{2.27}$$

This closed-form arc integration avoids the drift that a plain forward-Euler step introduces when the turning rate is large, and is the form that the implementation in `src/Adaptative_Shield/BicycleModel.py` uses.

2.7. CARLA SIMULATOR

CARLA [9] is an open-source simulator for autonomous driving research built on Unreal Engine 4. Its features relevant to this thesis are summarised below.

- **Client–server architecture.** The simulator runs as a server process (`CarlaUE4.exe`) that holds the world state and renders the scene, while the Python client connects to it over a TCP port. This decoupling makes it straightforward to run training and the simulator on different machines if desired.

- **Synchronous mode.** By default CARLA runs asynchronously, which is convenient for human interaction but non-deterministic for learning. When `synchronous=True` the server waits for a `world.tick()` call from the client before advancing one fixed time step (here, $\Delta t = 0.05\,\mathrm{s}$, i.e. $20\,\mathrm{Hz}$). This is a prerequisite for reproducible experiments.

- **Waypoint API.** The road network is represented internally as an OpenDRIVE map from which CARLA exposes a lattice of *waypoints* along the centre of every lane. Each waypoint carries metadata such as lane width, lane type, road curvature and the allowed lane-change directions. In this project the Waypoint API is the primary source of geometric information for both the reward shaper and the safety shields.

- **Sensors.** CARLA offers a library of simulated sensors (cameras, LIDAR, semantic segmentation, collision, lane invasion, GNSS, IMU). The framework uses three of them: a semantic LIDAR (see Chapter 4), a collision detector and a lane-invasion detector.

- **Traffic Manager.** Non-playable vehicles (NPCs) are controlled by an internal module called the *Traffic Manager*, which follows CARLA's own traffic rules. The framework uses it to instantiate the traffic density required by each curriculum stage.

- **Predefined towns.** CARLA ships with several maps. Town04 is a highway loop with entrance ramps and multi-lane segments; it is the main benchmarking map used in this thesis (Chapter 6).

2.8. RELATED WORK

2.8.0.0.1. Deep RL for autonomous driving.

The surveys by Kiran et al. [4] and Aradi [5] offer broad panoramas of the field. PPO is by far the most common backbone for continuous-control experiments on CARLA; notable examples include the work of Chen et al. [21]

on urban driving and the CARLA Leaderboard submissions that combine PPO with learned perception [22].

2.8.0.0.2. Safety shields in simulated driving.

Shielding has been applied to highway benchmarks such as HighwayEnv and the CommonRoad suite. Krasowski et al. [23] use reachability analysis to certify lane-change manoeuvres; Wabersich and Zeilinger [24] combine learning-based control with a model-predictive safety filter that projects every action onto the largest set in which recursive feasibility can be guaranteed; and MetaDrive [25] provides a lighter-weight shielding interface that inspired the early prototypes of this project. The present work contributes a *pair* of interchangeable shields in the CARLA ecosystem, which, to the author's knowledge, is less commonly explored than in HighwayEnv or MetaDrive.

2.8.0.0.3. Curriculum learning.

The general idea of presenting tasks to a learner in order of increasing difficulty dates back to Bengio et al. [26]. Applications to RL range from procedural domain randomisation [27] to automatic curriculum generation via self-play. The curriculum manager described in Section 4.7 of Chapter 4 is hand-designed around traffic density, but it includes a *rollback* mechanism that is uncommon in the curriculum-learning literature and that is motivated by the empirical instability observed on the simpler 0–20 NPC progression.

2.8.0.0.4. Bicycle model for trajectory prediction.

The kinematic bicycle model has been used as a forward model in both classical control [10] and recent learning-based approaches. Kong et al. [20] show that, at the speeds and curvatures typical of urban driving, the kinematic model is a close enough approximation of the dynamic bicycle model for planning horizons below one second. This justifies its use inside the adaptive shield, whose longest prediction horizon is ten steps of $0.05\,\mathrm{s}$ each, i.e. half a second.

3. REQUIREMENTS ANALYSIS

This chapter captures the requirements that guided the design of the framework. It distinguishes functional requirements (what the system has to *do*, Section 3.1) from non-functional requirements (how *well* it has to do it, Section 3.2). Section 3.3 lists the technical constraints imposed by the simulator, the hardware and the academic timeline of the project. Section 3.4 converts these requirements into measurable acceptance criteria that can be evaluated with the metrics logged by the training pipeline.

3.1. FUNCTIONAL REQUIREMENTS

Functional requirements describe the observable behaviour of the system. Each requirement is given a short identifier of the form `FR-XX`, a descriptive title and a succinct rationale. Requirements that depend on a specific implementation choice are justified in Chapter 4.

3.2. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe qualities of the system: how fast, how portable, how maintainable it must be. They are grouped below by ISO/IEC 25010 quality dimension.

3.3. TECHNICAL CONSTRAINTS

The requirements above must be satisfied inside the technical envelope described in Table 3.3. These constraints are not design decisions but external boundary conditions: the CARLA version is fixed by the research group's infrastructure, the hardware is the author's workstation, and the timeline is set by the Bachelor's thesis calendar.

3.4. ACCEPTANCE CRITERIA

Acceptance criteria translate the previous requirements into measurable conditions over the metrics produced by `main_train.py` and `main_eval.py`. They are used in Chapter 6 as the reference against which the three shielding configurations are compared. The baseline corresponds to the unconstrained PPO agent (`-shield_type none`).

The criteria with placeholder thresholds ($\alpha, \beta, \gamma, \tau$) are explicitly left to be fixed once the experiments of Chapter 6 have been executed. This keeps the requirements traceable while avoiding the pitfall of committing to numeric targets before the empirical evidence is available.

Table 3.1.: Functional requirements of CALA-SafeRL.

ID	Title	Description
FR-01	Gymnasium environment	The simulator shall be exposed through a `gymnasium.Env`-compatible wrapper supporting `reset`, `step`, `render` and `close`.
FR-02	Continuous control	The action space shall be a two-dimensional continuous box $[-1, 1]^2$ corresponding to steering and joint throttle–brake.
FR-03	Semantic LIDAR	The observation shall include three LIDAR channels (combined, dynamic, static) of 240 rays each, normalised to $[0, 1]$.
FR-04	Lane and route features	The observation shall include lane features (lateral offset, heading error, edge distances, curvature, lane-change direction) and route features (waypoint angles, progress, speed limit) derived from the Waypoint API.
FR-05	PPO training loop	A PPO agent with GAE, clipped surrogate, value loss, entropy bonus and KL early stopping shall be provided.
FR-06	Interchangeable shields	The training and evaluation entry points shall accept a `-shield_type` argument with values `none`, `basic` or `adaptive`, and switch shield implementations without code changes.
FR-07	Dense reward shaping	A wrapper shall augment CARLA's sparse reward with dense components (speed, lane centring, heading, smoothness, progress, idle, shield interaction).
FR-08	Performance-driven curriculum	A curriculum manager shall progressively add NPC traffic as the agent's performance improves and shall roll back when performance collapses.
FR-09	Checkpoint persistence	The agent shall save the best checkpoint on reward improvement and periodic checkpoints at a configurable frequency.
FR-10	Live metrics	Every episode and every PPO update shall be logged to a SQLite database in WAL mode so that an external dashboard can read it while training is still in progress.
FR-11	Evaluation dashboard	A headless evaluation mode and a Matplotlib dashboard mode shall be available, the latter showing LIDAR polar plot, speed gauge, lateral offset, shield status and intervention count.
FR-12	Lane-change awareness	Both the reward shaper and the shield shall detect the condition *lane change is permitted by road markings* and relax penalties that would otherwise block the manoeuvre.
FR-13	Reproducibility	Training and evaluation shall be seeded independently ($\mathrm{seed} = 42$ and $\mathrm{seed} = 100$ respectively) and the seeds shall be exposed through command-line flags.

Table 3.2.: Non-functional requirements of CALA-SafeRL.

ID	Dimension	Description
NFR-01	Performance	The simulator shall run in synchronous mode at $20\,\mathrm{Hz}$ ($\Delta t = 0.05\,\mathrm{s}$); all wrappers shall introduce an overhead small enough to sustain this rate on a single GPU workstation.
NFR-02	Determinism	Given a fixed seed and a fixed hardware/software configuration, two training runs shall produce the same episode outcomes up to floating-point non-associativity.
NFR-03	Modularity	The wrapper chain shall obey the Open/Closed principle: adding a new shield or a new reward component shall require no modifications to the existing layers.
NFR-04	Maintainability	Python source files shall pass `ruff check` with the project's default configuration; public classes shall carry short docstrings.
NFR-05	Portability	The framework shall run on Linux and Windows and shall automatically select the best PyTorch device among CUDA, Apple MPS and CPU.
NFR-06	Observability	Every component that modifies actions or rewards shall expose an auxiliary `info` dictionary key (`executed_action`, `shield_activated`, `risk_level`, reward sub-components) so that training behaviour can be diagnosed post-hoc.
NFR-07	Extensibility	The live-metrics schema shall admit new keys without breaking existing dashboards; both the SQLite logger and the Streamlit reader shall treat unknown metrics as first-class citizens.
NFR-08	Testability	The reward shaper shall ship with deterministic unit tests that do not require a running CARLA server.
NFR-09	Documentation	The project shall include a `CLAUDE.md` file that documents the wrapper chain, the observation space layout and the commands to reproduce training and evaluation.

Table 3.3.: Technical constraints of the project.

Category	Constraint
Simulator	CARLA 0.9.16 running on Windows with the `-RenderOffScreen` flag and fixed port 2000.
Language	Python ≥ 3.10.
Core libs	PyTorch (GPU when available), Gymnasium, NumPy, Matplotlib, Streamlit, SQLite. Licences shall be compatible with open academic distribution (MIT, BSD-3, Apache-2.0).
Hardware	Single desktop workstation; GPU with at least $6\,\mathrm{GB}$ of VRAM is required to run CARLA and the PyTorch policy concurrently. Training on CPU is supported but orders of magnitude slower.
Maps	The main benchmark shall be Town04. Additional towns (Town01–Town07, Town10HD) may be used for qualitative evaluation but are not guaranteed to converge with the default hyperparameters.
Timeline	The project shall be delivered within the Bachelor's thesis calendar of the Universidad de Deusto. This precludes large hyperparameter sweeps that would require multiple weeks of simulator time per configuration.
Safety	The framework shall only run inside simulation; deployment on a physical vehicle is explicitly out of scope.

Table 3.4.: Acceptance criteria derived from the functional and non-functional requirements. Thresholds are placeholders to be replaced by the empirical numbers obtained in Chapter 6.

ID	Metric	Acceptance threshold
AC-01	Reward convergence	The 100-episode rolling average reward of the shielded agent shall be no worse than the baseline after an equal amount of training.
AC-02	Crash rate	With the adaptive shield, the 100-episode rolling `crash_rate` shall drop by at least a factor of α with respect to the baseline ($\alpha =$ TBD in Chapter 6).
AC-03	Off-road rate	Under any shield, the 100-episode rolling `offroad_rate` shall remain below a threshold β ($\beta =$ TBD).
AC-04	Success rate	On 10 evaluation episodes with seed 100, the agent shall reach the `success_distance` target in at least γ of them ($\gamma =$ TBD).
AC-05	Shield interference	The fraction of steps in which the adaptive shield activates shall be bounded by τ in the *safe* risk level, 2τ in *warning* and 5τ in *critical* ($\tau =$ TBD).
AC-06	Speed compliance	At least 90% of steps with a valid speed limit shall satisfy speed_kmh $\leq 1.05 \cdot$ speed_limit_kmh.
AC-07	Curriculum progression	The curriculum manager shall advance from stage 0 to at least stage 3 during a standard $2\,500$-episode run in all shielded configurations.
AC-08	Dashboard liveness	The Streamlit dashboard shall refresh training metrics with an end-to-end latency below $2\,\mathrm{s}$ from the moment the logger writes a row to the SQLite database.
AC-09	Reproducibility	Two independent runs with the same seed and configuration shall produce identical reward curves in the first 50 episodes (before floating-point divergence becomes significant).

4. SYSTEM DESIGN

This chapter describes how the concepts introduced in Chapter 2 are instantiated into a concrete software architecture. Section 4.1 presents the layered wrapper chain, Sections 4.2–4.3 formalise the observation and action spaces, Section 4.4 describes the PPO agent, Section 4.5 the two safety shields, Section 4.6 the reward shaper, Section 4.7 the curriculum manager and Section 4.8 the live metrics pipeline.

4.1. ARCHITECTURAL OVERVIEW

The framework is organised as a vertical stack of components that communicate through the standard `gymnasium.Env` interface. Each layer consumes the observation produced by the layer below and (possibly) transforms both the action travelling downwards and the reward travelling upwards. Figure 4.1 shows the chain.

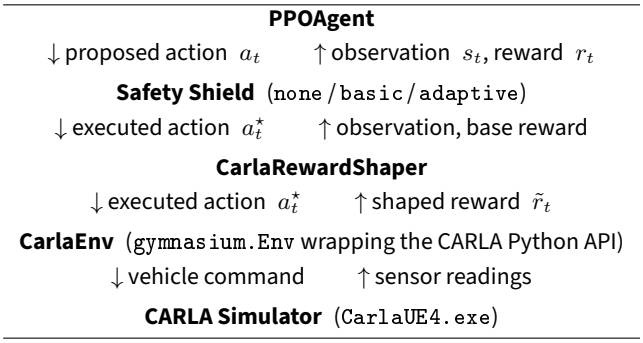

Figure 4.1.: Wrapper chain of the framework. Each layer is a `gymnasium.Wrapper` that can be composed, replaced or removed without touching its neighbours.

A subtlety of this layout deserves an explicit comment. The safety shield is placed *above* the reward shaper, not below it. This ordering has a functional reason: the shaper reads the `info` keys `executed_action` and `proposed_action`—both produced by the shield—to compute components such as the shield-intervention penalty, the smoothness penalty during an override and the lane-change transition guard. If the shaper were placed above the shield, those keys would not yet exist when the shaper inspects the `info` dictionary, and the shield interaction could not be accounted for in the reward.

The environment runs in *synchronous mode* with a fixed time step of $\Delta t = 0.05\,\mathrm{s}$. Every call to `env.step(action)` sends the action to CARLA, requests a single `world.tick()` and collects the resulting sensor readings. This guarantees determinism given a fixed seed, at the cost of not being able to use CARLA's

asynchronous rendering speed-ups.

4.2. OBSERVATION SPACE

The observation vector $s_t \in \mathbb{R}^{735}$ concatenates three semantic LIDAR channels and a small block of geometric and route features. Appendix B contains the full index-by-index table; this section describes the design rationale.

4.2.1. Semantic LIDAR

The framework uses a simulated LIDAR with $N = 240$ rays uniformly distributed over $360°$ (angular resolution $1.5°$/ray, range $50\,\mathrm{m}$). Each ray is normalised to the $[0, 1]$ interval, where 1 denotes a free ray up to the maximum range and values close to 0 denote an obstacle next to the vehicle. The semantic channel of each ray is obtained from CARLA's `SemanticLidarSensor`, which tags every hit with the `carla.CityObjectLabel` of the struck primitive.

Three derived channels are packed into the observation:

- obs$[0 : 240]$ — **combined channel**. Includes every obstacle (dynamic and static) plus road edges.
- obs$[240 : 480]$ — **dynamic-only channel**. Keeps only rays that struck a vehicle or a pedestrian.
- obs$[480 : 720]$ — **static-only channel**. Keeps only rays that struck a wall, a barrier, a pole or a similar static object.

The decomposition is motivated by an empirical observation: a single-channel LIDAR induces a *zig-zag* pattern when the vehicle drives along a guardrail, because the policy interprets the low returns from the guardrail as a moving obstacle and keeps trying to evade it laterally. Separating dynamic from static obstacles allows the policy to learn that static obstacles can be treated as soft boundaries while dynamic obstacles require anticipation.

4.2.2. Lane, vehicle and route features

A 15-dimensional vector block is appended to the LIDAR. Its components are:

- obs$[720 : 728]$ — **lane features (8 dims)**: `lateral_offset_norm`, `heading_error_norm`, `on_edge_warning`, `lane_width_norm`, `dist_left_edge_norm`, `dist_right_edge_norm`, `lane_change_permitted`, `road_curvature_norm`.
- obs$[728 : 730]$ — **vehicle state (2 dims)**: `speed_norm`, `steering`.
- obs$[730 : 735]$ — **route features (5 dims)**: `next_wp_angle_norm` ($5\,\mathrm{m}$ ahead), `wp_angle_20m_norm`, `progress_norm`, `speed_limit_norm`, `speed_ratio`.

All features are extracted from CARLA's Waypoint API and therefore carry centimetre-level accuracy. The two offsets `dist_left_edge_norm` and `dist_right_edge_norm` are normalised by the lane width, so that both take the value 0.5 when the vehicle is at the centre of the lane and approach 0 as it nears the corresponding edge.

4.3. ACTION SPACE

The action space is the continuous box $\mathcal{A} = [-1, 1]^2$. Its two components are

- $a^{(1)} \in [-1, 1]$ — *steering*, mapped linearly to $\delta \in [-\delta_{\max}, +\delta_{\max}]$ with $\delta_{\max} = 0.6\,\mathrm{rad}$.
- $a^{(2)} \in [-1, 1]$ — *joint throttle–brake*: positive values drive the throttle, negative values drive the brake, zero is coast. The acceleration saturation values are $+3.0\,\mathrm{m/s^2}$ and $-7.0\,\mathrm{m/s^2}$.

Merging throttle and brake into a single signed channel has two practical advantages: it reduces the dimensionality of the policy output, and it prevents the agent from learning the counter-productive behaviour of activating throttle and brake simultaneously.

Continuous actions are sampled from a diagonal Gaussian and squashed through a hyperbolic tangent to keep them inside the box:

$$u_t \sim \mathcal{N}\big(\mu_\theta(s_t),\, \sigma_\theta^2\big), \qquad a_t = \tanh(u_t). \tag{4.1}$$

The log probability of the squashed sample requires the log-determinant of the tanh Jacobian [28]:

$$\log \pi_\theta(a_t \mid s_t) \;=\; \log \mathcal{N}\big(u_t;\, \mu_\theta(s_t),\, \sigma_\theta\big) \;-\; \sum_i \log\big(1 - a_{t,i}^2 + \varepsilon\big), \tag{4.2}$$

where ε is a small constant added for numerical stability. The same correction is used to evaluate the log probability of an executed action, which is a subtle but important point for credit assignment under a shield

(Section 5.4.1).

4.4. AGENT DESIGN

4.4.1. Actor–Critic network

The actor and the critic share a representation of the observation but have separate heads. Feeding the raw 735-dimensional observation directly into a shallow MLP dilutes the signal: the 720 LIDAR dimensions dominate the input while the 15 lane/vehicle/route features carry most of the task-relevant information. The network therefore includes a dedicated *LIDAR encoder*:

$$\mathrm{enc}(s_t) \;=\; \mathrm{Tanh}\Big(W_2\,\mathrm{Tanh}\big(W_1\,s_t^{\mathrm{LIDAR}} + b_1\big) + b_2\Big) \;\in\; \mathbb{R}^{64}, \tag{4.3}$$

with $W_1 \in \mathbb{R}^{256\times 720}$ and $W_2 \in \mathbb{R}^{64\times 256}$. The encoder output is concatenated with the 15-dimensional vector block to produce a 79-dimensional feature $\phi(s_t)$:

$$\phi(s_t) \;=\; \big[\,\mathrm{enc}(s_t)\;;\; s_t^{\mathrm{vector}}\,\big] \in \mathbb{R}^{79}. \tag{4.4}$$

The critic is a two-layer MLP with 256 units per hidden layer and a scalar output:

$$V_\theta(s_t) \;=\; W^V\,\mathrm{Tanh}\Big(\mathrm{MLP}_{\mathrm{critic}}\big(\phi(s_t)\big)\Big) + b^V. \tag{4.5}$$

The actor shares the same trunk layout but, instead of a scalar output, produces a two-dimensional mean vector and has a separate log-standard-deviation parameter that does not depend on the state:

$$\mu_\theta(s_t) \;=\; W^\mu\,\mathrm{Tanh}\Big(\mathrm{MLP}_{\mathrm{actor}}\big(\phi(s_t)\big)\Big) + b^\mu, \qquad \log\sigma_\theta \in \mathbb{R}^2. \tag{4.6}$$

The log-standard-deviation is clamped to the range $[-3.0,\,-0.2]$. Its initial value is set to -0.7, which corresponds to a pre-$\tanh$ standard deviation of approximately 0.50. This gives broad but non-saturating exploration at the beginning of training.

4.4.2. PPO update

At each environment step the agent appends the tuple $\left(s_t,\, a_t^\star,\, \log \pi_\theta(a_t^\star \mid s_t),\, r_t,\, d_t,\, \mathrm{trunc}_t,\, V_t^{\text{boot}}\right)$ to the rollout buffer. Two details are specific to this project:

1. The buffer stores the *executed* action $a_t^\star$ (the action that actually reached the simulator), not the action sampled from the policy. The log probability stored in the buffer is recomputed by `agent.evaluate_action` with Equation (4.2). This is what ties the gradient computation to the action the environment saw, avoiding the credit-assignment error that would otherwise be introduced by the shield.

2. When an episode is truncated by a timeout but not terminated (no crash, no off-road), the value $V_t^{\text{boot}} = V_\theta(s_{t+1})$ is stored and used to bootstrap GAE at that step. When the episode terminates naturally, $V_t^{\text{boot}} = 0$ is stored.

When the buffer reaches $M = 2048$ samples, GAE is run backwards through the rollout (Equation (2.15)), advantages are standardised to zero mean and unit variance, and the return target is divided by the running standard deviation described in Section 4.4.4. A loop of up to $K = 10$ epochs of PPO is then performed, with an early stop triggered by Equation (2.12) once the moving average of the approximate KL divergence exceeds $D_{\mathrm{KL}}^\star$. The gradient is clipped to an ℓ_2 norm of 1.0 and the learning rate follows the cosine schedule of Equation (2.13).

4.4.3. Observation normalisation

Observation normalisation is performed online with Welford's incremental mean/variance estimator [29]. Given a running mean $\hat{\mu}$, running variance $\hat{\sigma}^2$ and count N, the update for a batch $\mathbf{x}$ of size B is

$$\Delta = \bar{\mathbf{x}} - \hat{\mu}, \tag{4.7}$$

$$\hat{\mu} \leftarrow \hat{\mu} + \Delta \frac{B}{N+B}, \tag{4.8}$$

$$\hat{\sigma}^2 \leftarrow \frac{N\hat{\sigma}^2 + B\,\mathrm{Var}(\mathbf{x}) + \Delta^2 \frac{NB}{N+B}}{N+B}, \tag{4.9}$$

$$N \leftarrow N + B. \tag{4.10}$$

Normalised observations are clipped to $[-5, 5]$ to contain outliers.

An implementation detail that turned out to be important is that normalisation is applied *only* to the 15-dimensional vector block, not to the 720 LIDAR dimensions. The LIDAR is already in $[0, 1]$ by construction, so

normalisation would not change its scale but would artificially introduce outliers when the curriculum advances from stage 0 (no NPCs, the dynamic channel is the constant 1.0) to stage 1 (dynamic channel suddenly has non-unit entries). Keeping the LIDAR un-normalised avoids this distribution shift and therefore avoids the gradient spikes observed in early prototypes.

4.4.4. Return normalisation

The return targets passed to the critic loss in Equation (2.10) can span several orders of magnitude, which makes MSE training unstable. To counter this, the returns are divided by the running standard deviation

$$\hat{R}_t \leftarrow \frac{\hat{R}_t}{\hat{\sigma}_R + \varepsilon}, \tag{4.11}$$

where $\hat{\sigma}_R$ is computed by the same Welford estimator applied to the rollout returns. Note that the mean is *not* subtracted; subtracting a running mean of returns would bias the advantages, which are already standardised separately.

4.5. SAFETY SHIELDS

Both shields implement the function Σ of Equation (2.17). They differ in the semantic information they consume and in the strategy by which they generate the corrected action.

4.5.1. Basic shield

The basic shield (`CarlaSafetyShield`) operates in two layers. The first layer analyses the LIDAR scan for imminent obstacles; the second layer uses the Waypoint API to detect unsafe lane-keeping conditions.

4.5.1.0.1. Obstacle analysis.

Let $\mathrm{scan}_t \in [0,1]^{240}$ be the combined LIDAR channel at time t. The shield partitions it into three angular sectors:

$$\mathrm{front}_t = \mathrm{scan}_t\big[225^\circ\!:\!315^\circ\big] \cup \mathrm{scan}_t\big[0^\circ\!:\!22.5^\circ\big] \cup \mathrm{scan}_t\big[337.5^\circ\!:\!360^\circ\big], \tag{4.12}$$

$$\mathrm{side}_t^{\mathrm{R}} = \mathrm{scan}_t\big[60^\circ\!:\!120^\circ\big], \tag{4.13}$$

$$\mathrm{side}_t^{\mathrm{L}} = \mathrm{scan}_t\big[240^\circ\!:\!300^\circ\big], \tag{4.14}$$

and declares the situation unsafe whenever any of the following conditions holds:

$$\min \mathrm{front}_t < \tau_{\mathrm{f}}, \quad \min \mathrm{side}_t^{\mathrm{R}} < \tau_{\mathrm{s}}, \quad \min \mathrm{side}_t^{\mathrm{L}} < \tau_{\mathrm{s}}. \tag{4.15}$$

The default thresholds are $\tau_{\mathrm{f}} = 0.15$ and $\tau_{\mathrm{s}} = 0.04$ in normalised LIDAR units.

4.5.1.0.2. Lane-keeping analysis.

The shield reads the lateral offset ℓ_t = `lateral_offset_norm` and the heading error h_t = `heading_error_norm` from the `info` dict and declares the situation unsafe when

$$|\ell_t| > \tau_\ell \quad \text{or} \quad |h_t| > \tau_h, \tag{4.16}$$

with defaults $\tau_\ell = 0.82$ (expressed as a fraction of the lane half-width) and $\tau_h = 0.60$.

4.5.1.0.3. Correction law.

When the situation is unsafe the shield overrides the action with a proportional controller. Let k_{s} and k_{b} be the steering and brake gains. The correction is

$$\delta_t^\star \;=\; \mathrm{clip}\Big(-\,k_{\mathrm{s}}\,\ell_t \;-\; 0.45\,k_{\mathrm{s}}\,h_t \;+\; \mathrm{sidepush}_t,\; -1,\; 1 \Big), \tag{4.17}$$

$$(tb)_t^\star \;=\; \mathrm{clip}\Big(-\;k_{\mathrm{b}}\;\max\big(0,\; \tau_{\mathrm{f}} - \min \mathrm{front}_t\big)/\tau_{\mathrm{f}},\; -1,\; 0 \Big), \tag{4.18}$$

where $\mathrm{sidepush}_t$ contains the lateral correction triggered by close obstacles in $\mathrm{side}^{\mathrm{R}}$ or $\mathrm{side}^{\mathrm{L}}$. Equations (4.17)–(4.18) follow the *minimal-interference principle*: if ℓ_t, h_t and $\min \mathrm{front}_t$ are all far from their thresholds, all three contributions vanish and the shield produces the zero-override action. Conversely, the closer the system is to an unsafe region, the stronger the corrective action.

4.5.2. Adaptive-horizon shield

The adaptive-horizon shield (`CarlaAdaptiveHorizonShield`) extends the basic design with three mechanisms: a risk classifier, a trajectory prediction with the bicycle model, and a priority-ordered candidate search.

4.5.2.0.1. Risk classifier.

Three risk levels are defined: safe, $\mathrm{warning}$ and $\mathrm{critical}$. Each level fixes a verification horizon H (number of bicycle-model steps) and a threshold multiplier κ applied to the basic thresholds. The configuration is

$$\mathrm{safe}: \ H = 1, \ \kappa = 1.0; \qquad \mathrm{warning}: \ H = 5, \ \kappa = 1.5; \qquad \mathrm{critical}: \ H = 10, \ \kappa = 2.0.$$

The level is chosen as the worst of two independent criteria. The first is the *frontal-distance* criterion: let $d_t^{\mathrm{dyn}} = \min \mathrm{front}_t^{\mathrm{dynamic}}$ be the minimum of the dynamic-only LIDAR front sector. Then

$$\mathrm{level}_t^{\mathrm{frontal}} = \begin{cases} \mathrm{safe} & \text{if } d_t^{\mathrm{dyn}} > 0.50, \\ \mathrm{warning} & \text{if } 0.20 < d_t^{\mathrm{dyn}} \leq 0.50, \\ \mathrm{critical} & \text{if } d_t^{\mathrm{dyn}} \leq 0.20. \end{cases} \tag{4.19}$$

The second criterion, *lateral position*, thresholds $|\ell_t|$ at 0.70 and 0.85 (warning and critical, respectively). If the Waypoint API reports that a lane change is permitted at the current location, a *critical* lateral level is downgraded to *warning* so that the shield does not block a legitimate lane-change manoeuvre.

4.5.2.0.2. Trajectory check.

Given a candidate action $a = (\delta, tb)$, the shield asks the bicycle model of Section 2.6 for the trajectory $\{(x_k, y_k, \theta_k)\}_{k=0}^{H}$. Each predicted position (x_k, y_k) is then projected onto the road using `carla.Map.get_waypoint` restricted to driving lanes. The candidate is declared safe when all of the following hold:

$$\min \mathrm{front}_t^{\mathrm{combined}} \ \geq \ \tau_{\mathrm{f}}/\kappa, \qquad \min \mathrm{side}_t^{\mathrm{R/L,\,static}} \ \geq \ \tau_{\mathrm{s}}/\kappa, \tag{4.20}$$

$$\mathrm{nearest\_pedestrian\_m}_t \ \geq \ 4.0\,\mathrm{m}, \tag{4.21}$$

$$\forall\, k \in \{1, \ldots, H\}: \ |\ell_k| \ \leq \ \max\bigl(\tau_\ell/\kappa, \ 0.45\bigr), \tag{4.22}$$

where $\ell_k = \texttt{lat\_offset}(x_k, y_k)/(\texttt{lane\_width}/2)$ is the normalised lateral offset of the predicted position with respect to the nearest driving lane. The floor of 0.45 in Equation (4.22) prevents the lateral threshold from becoming so tight in the *critical* level that no correction action can satisfy it. When the *lane-change permitted* flag is set, the lateral threshold is further raised to 1.2 so that the transition band between lanes is admissible.

4.5.2.0.3. Priority-ordered candidate search.

If the agent's action a_t does not pass the trajectory check, the shield inspects an ordered list of candidate actions and returns the first one that does. The ordering depends on the dominant threat, which is identified in three mutually exclusive buckets:

- **Pedestrian emergency** ($\text{nearest\_pedestrian\_m}_t < 4\,\text{m}$). The override is always $(\delta, tb) = (0, -1)$ (maximum brake, no search).
- **Dynamic threat** (nearby vehicle in front, predicted time-to-contact below $\max(5\,\text{s},\ 1.5\,v_t)$ metres). The first candidates privilege braking; steering is adjusted afterwards.
- **Static threat** (static obstacle in the side sector). The first candidates privilege a steering correction back to the lane centre, accompanied by a mild brake.
- **Lateral-only threat** ($|\ell_t| > 0.65$ without any dynamic or static threat). The first candidates apply a lane-centring steering with a small throttle to keep enough speed for directional control.
- **Balanced fallback** (default): a symmetric list of brake/steer combinations.

If none of the candidates pass the trajectory check, the shield returns the *fallback* action

$$a_t^{\text{fallback}} = \big(\text{clip}(-1.2\,\ell_t,\ -0.8,\ 0.8),\ -0.4\big). \tag{4.23}$$

4.5.3. Bicycle-model parameters

The bicycle model described in Section 2.6 is instantiated with the parameters of CARLA's Tesla Model 3, summarised in Table 4.1.

Table 4.1.: Parameters of the kinematic bicycle model used by the adaptive shield.

Symbol	Value	Meaning
L	$2.87\,\text{m}$	Wheelbase of the Tesla Model 3
$\delta_{\max}$	$0.60\,\text{rad}\ (\approx 34^\circ)$	Maximum steering angle
Δt	$0.05\,\text{s}$	Time step, matched to the simulator
$a_{\max}$	$3.0\,\text{m/s}^2$	Maximum forward acceleration
$-a_{\min}$	$7.0\,\text{m/s}^2$	Maximum deceleration (brake)

4.6. REWARD SHAPING

The reward shaper `CarlaRewardShaper` produces a shaped reward $\tilde{r}_t$ by augmenting the sparse base reward r_t^{base} of `CarlaEnv` (success $+30$, collision -10, off-road -20) with twelve dense components. The shaped reward is

$$\tilde{r}_t \;=\; r_t^{\text{base}} + r_t^{\text{alive}} + r_t^{\text{speed}} + r_t^{\text{lane}} + r_t^{\text{head}} + r_t^{\text{prog}} + r_t^{\text{no-shield}} - p_t^{\text{smooth}} - p_t^{\text{inv}} - p_t^{\text{edge}} - p_t^{\text{wrong-h}} - p_t^{\text{shield}} - p_t^{\text{idle}} - p_t^{\text{drift}}. \tag{4.24}$$

Each component is described below. Unless stated otherwise, all variables read from the `info` dictionary come from the Waypoint API.

4.6.0.0.1. Speed gate.

A common factor $g_t \in [0,1]$ modulates the components that only make sense when the vehicle is actually moving. It is a linear ramp of the speed v_t between the two thresholds $v_{\min}$ and v_g:

$$g_t \;=\; \text{clip}\left(\frac{v_t - v_{\min}}{v_g - v_{\min}},\, 0,\, 1\right). \tag{4.25}$$

With defaults $v_{\min} = 5\,\text{km/h}$ and $v_g = 10\,\text{km/h}$.

4.6.0.0.2. Alive bonus.

$$r_t^{\text{alive}} \;=\; c_{\text{alive}}\, g_t \;\cdot\; \mathbb{1}[\,\text{on\_road}_t\,]. \tag{4.26}$$

Gating by g_t is essential to prevent the local optimum *"stop and collect 0.15/step"* that plagued earlier prototypes of the shaper.

4.6.0.0.3. Speed bonus.

Let v_t^{lim} be the effective speed limit (from CARLA's speed limits or the project-level target, whichever is defined). A curvature correction reduces the target speed in sharp bends:

$$v_t^{\text{target}} \;=\; v_t^{\text{lim}} \cdot \max\Big(\,1 - c_{\text{curv}} \cdot \min\big(|\kappa_t|/0.6,\, 1\big),\; 0.4\,\Big). \tag{4.27}$$

The speed bonus is a Gaussian centred on this curvature-adjusted target, with a speed-ceiling penalty when the vehicle exceeds the raw limit by more than the margin:

$$r_t^{\text{speed}} \;=\; w_{\text{v}}\,\exp\!\left(-\frac{(v_t - v_t^{\text{target}})^2}{2\sigma^2}\right) \;-\; w_{\text{v}}\cdot 0.8\,\max\!\Big(0,\;\frac{v_t - (1+\eta)\,v_t^{\text{lim}}}{v_t^{\text{lim}}}\Big), \tag{4.28}$$

with $\sigma = 0.35\,v_t^{\text{target}}$ and $\eta = 0.05$ the tolerance margin.

4.6.0.0.4. Lane centring and heading.

$$r_t^{\text{lane}} \;=\; w_{\text{lc}}\,\max(g_t,\,0.3)\,\text{clip}\big(\,\min(d_t^{\text{L}}, d_t^{\text{R}})/0.5,\,0,\,1\big), \tag{4.29}$$

$$r_t^{\text{head}} \;=\; w_{\text{h}}\,g_t\,\exp\!\left(-\frac{h_t^2}{2\cdot 0.40^2}\right), \tag{4.30}$$

where d_t^{L}, d_t^{R} are the normalised distances to the left and right lane edges. The floor of 0.3 in Equation (4.29) provides a small centring signal even when the vehicle is stopped, which helps the policy learn to align before accelerating.

4.6.0.0.5. Smoothness penalty.

$$p_t^{\text{smooth}} \;=\; w_{\text{sm}}\,\big|\delta_t - \delta_{t-1}\big|. \tag{4.31}$$

4.6.0.0.6. Invasion, edge and wrong-heading penalties.

The lane-invasion penalty uses the signal of CARLA's lane-invasion sensor; it is weighted by the invasion severity (a function of $|\ell_t|$) and is suppressed when the invasion is intentional (large steering magnitude) or when a lane change is permitted. The edge penalty is a quadratic ramp triggered when the closest edge distance drops below 0.4; the wrong-heading penalty triggers when $|\text{heading\_error\_deg}_t|$ exceeds 90°.

4.6.0.0.7. Progress bonus.

A fixed bonus w_{prog} is awarded every time the vehicle crosses a distance milestone of $\Delta_{\text{prog}} = 25\,\text{m}$. This avoids the pathological behaviour of a speed bonus that rewards high speed regardless of whether the vehicle is actually moving along the route.

4.6.0.0.8. Shield interaction.

Let a_t be the action proposed by the agent and $a_t^\star$ the action executed by the environment. Define the shield-divergence measure

$$\Delta_t^{\text{shield}} \;=\; \frac{\|a_t^\star - a_t\|_2}{2\sqrt{2}}. \tag{4.32}$$

The normalisation by $2\sqrt{2}$ maps the divergence to the unit interval (the maximum distance between two points of $[-1,1]^2$ is $2\sqrt{2}$). The shield-interaction components are

$$p_t^{\text{shield}} \;=\; \Delta_t^{\text{shield}}\, w_{\text{sh}}\, \mathbb{1}[\,\text{shield\_activated}_t\,], \qquad r_t^{\text{no-shield}} \;=\; w_{\text{no-sh}}\, \mathbb{1}[\,\neg\text{shield\_activated}_t\,]. \tag{4.33}$$

Either w_{sh} or $w_{\text{no-sh}}$ can be zero, which makes it possible to penalise shield activations, to reward the agent for not needing the shield, or to run in a neutral configuration.

4.6.0.0.9. Idle penalty and shield grace period.

An idle penalty discourages the agent from stopping. To prevent the penalty from punishing legitimate shield-induced stalls, a *grace period* of G steps is opened the first time the shield activates in an episode. The counter is re-armed *only* when the shield fires and the grace period has already expired, which prevents an always-active shield from suppressing the idle penalty forever.

4.6.0.0.10. Drift penalty.

Asymmetric driving (e.g. hugging the right edge) is penalised by

$$p_t^{\text{drift}} \;=\; w_{\text{dr}}\, \max\!\big(0,\, |d_t^{\text{L}} - d_t^{\text{R}}| - 0.3\big)\, \mathbb{1}\big[\min(d_t^{\text{L}}, d_t^{\text{R}}) < 0.35\big]\,. \tag{4.34}$$

4.6.0.0.11. Lane-change transition guard.

When the Waypoint API reports `lane_change_permitted` and $|\ell_t| > 0.5$, the penalties p_t^{inv}, p_t^{edge} and p_t^{drift} are all set to zero. This keeps the cost of a legitimate lane-change manoeuvre from blocking the behaviour during training.

The default weights of all components are listed in Appendix A, Table A.3.

4.7. CURRICULUM MANAGER

The curriculum manager divides the interval $[0, N_{\max}]$ of NPC vehicles uniformly in five stages: $(0,\, N_{\max}/4,\, N_{\max}/2,\, 3N_{\max}/4,\, N_{\max})$. For the default $N_{\max} = 40$ this yields $(0,\, 10,\, 20,\, 30,\, 40)$. The manager keeps two counters: the number of episodes at the current stage E_{stage} and a decay-weighted rollback counter C_{rb}.

4.7.0.0.1. Advancement rule.

After $E_{\text{stage}} \geq E^{\star} = 100$ episodes, if the stage-specific condition of Table 4.2 is satisfied, the manager advances one stage and resets both counters to zero.

4.7.0.0.2. Rollback rule.

After $E_{\text{stage}} > E^{\star}$, on every episode the counter C_{rb} is incremented by one if the stage is *collapsing* and decremented by one otherwise (but not below zero). When $C_{\text{rb}} \geq C^{\star} = 50$, the manager rolls back one stage and resets all counters. The decay by -1 on non-collapsing episodes prevents a streak of bad luck from triggering a rollback that is not warranted by a sustained performance drop.

Table 4.2.: Advancement and rollback thresholds per curriculum stage. All rates are 100-episode rolling averages.

Stage	NPCs	Advance if	Rollback if
0	0	offroad_rate $< 0.20 \wedge$ avg_reward > 0	—
1	$N_{\max}/4$	crash_rate $< 0.25 \wedge$ offroad_rate < 0.15	crash_rate $> 0.45 \vee$ offroad_rate > 0.40
2	$N_{\max}/2$	crash_rate < 0.15	crash_rate > 0.40
3	$3N_{\max}/4$	crash_rate < 0.10	crash_rate > 0.35
4	$N_{\max}$	— (terminal)	crash_rate > 0.30

4.7.0.0.3. Non-disruptive NPC update.

When the stage changes, the manager does *not* rebuild the environment. It updates the attribute `CarlaEnv.num_npc_vehicles` in place; the change takes effect at the next `env.reset()`. This avoids the deadlock observed in earlier prototypes that had to tear down the wrapper chain every time the stage changed.

4.8. LIVE METRICS PIPELINE

Training metrics are written to a per-run SQLite database at `runs/<run_name>/metrics.sqlite`. The logger (`src/Metrics/live_metrics.py`) opens the database in *Write-Ahead Logging* (WAL) [30] mode, which allows concurrent reads and writes: the training process writes while the Streamlit dashboard reads.

Two tables are used, corresponding to the two time axes of the experiment:

- `metrics_episode`: one row per episode, indexed by episode number. Columns include the reward components, the safety rates, the curriculum stage, and the LIDAR-derived minimum distances.
- `metrics_update`: one row per PPO update, indexed by the cumulative timestep. Columns include the policy loss, the value loss, the approximate KL, the number of epochs that actually ran, the gradient norm and the current learning rate.

The dashboard (`dataVisualizer.py`) reads both tables every few seconds and renders interactive Plotly charts. Unknown metric keys are rendered as soft-typed line plots, which keeps the dashboard forward-compatible with future additions to the logger.

5. IMPLEMENTATION

This chapter describes how the design of Chapter 4 is materialised in Python code. Section 5.1 introduces the technology stack and Section 5.2 the project layout. The subsequent sections cover each component: the CARLA wrapper (§5.3), the PPO agent (§5.4), the two shields (§5.5), the reward shaper (§5.6), the curriculum manager (§5.7), the entry-point pipelines (§5.8) and the reproducibility concerns (§5.9).

5.1. TECHNOLOGY STACK

The framework is written in pure Python (≥ 3.10) and relies on the libraries summarised in Table 5.1. Licences are reported because the framework is meant to be distributed as open academic software.

Table 5.1.: Main third-party dependencies.

Library	Role	Licence
`carla`	Python binding for CARLA 0.9.16	MIT
`gymnasium`	Standard RL environment API	MIT
`torch`	Deep learning backend	BSD-3
`numpy`	Numerical arrays	BSD-3
`matplotlib`	Evaluation dashboard	PSF-based
`streamlit`	Live training dashboard	Apache-2.0
`plotly`	Interactive charts in the dashboard	MIT
`sqlite3` (stdlib)	Metrics database	Public domain
`ruff`	Linter and formatter	MIT
`pytest`	Unit testing of the reward shaper	MIT

5.2. PROJECT LAYOUT

The repository root is organised as follows (directories with no relevance to the thesis are omitted):

- `main_train.py` — training entry point.
- `main_eval.py` — evaluation entry point.
- `dataVisualizer.py` — Streamlit dashboard that reads the SQLite metrics database.
- `src/CARLA/Env/carla_env.py` — base Gymnasium environment.
- `src/PPO/` — `ActorCritic.py`, `ppo_agent.py`, `RunningMeanStd.py`.

- `src/safety_shield.py` — basic safety shield.
- `src/Adaptative_Shield/` — `adaptive_horizon_shield.py`, `BicycleModel.py`.
- `src/reward_shaper.py` — dense reward shaper.
- `src/curriculumManager.py` — performance-driven curriculum.
- `src/Metrics/` — live metrics logger and offline evaluation reporter.
- `tests/test_reward_balance.py` — unit tests on the reward shaper that do not require a running CARLA server.
- `data/models/` — serialised checkpoints.
- `runs/` — per-run SQLite databases and TensorBoard logs.

Every module exposes either a class that inherits from `gymnasium.Wrapper` (the shields, the reward shaper) or a small self-contained helper (the curriculum, the running-mean estimator). There is no inter-module state other than what flows through the Gymnasium interface. This design makes the individual components trivially testable.

5.3. ENVIRONMENT WRAPPER

The class `CarlaEnv` inherits from `gymnasium.Env` and encapsulates the connection to the CARLA server. At construction time it performs the following setup:

1. Opens a TCP connection on `(host, port)` and loads the requested map.
2. Enables synchronous mode with `fixed_delta_seconds = 0.05`.
3. Instantiates the *Traffic Manager* on `tm_port` and spawns the requested number of NPC vehicles.
4. Sets the requested weather preset.
5. Registers the collision, lane-invasion and semantic-LIDAR sensors.
6. Defines `observation_space` as a Box$([0,1]^{735})$ and `action_space` as a Box$([-1,1]^{2})$.

On every `step(action)` call, the class converts the normalised action into a `carla.VehicleControl`, ticks the simulator exactly once, reads the sensors, computes the geometric features from the Waypoint API, builds the 735-dimensional observation, and derives the base reward together with the `done` and `truncated` flags. The `info` dictionary returned by `step` contains every scalar used by the downstream wrappers: `speed_kmh`, `speed_limit_kmh`, `lateral_offset_norm`, `heading_error_norm`, `dist_left_edge_norm`, `dist_right_edge_norm`, `road_curvature_norm`, `lane_change_permitted`, `on_road`, `collision`, `arrive_dest`, `out_of_road`, `nearest_vehicle_m`, `nearest_pedestrian_m` and `total_distance`, among others.

5.4. PPO AGENT

The PPO agent is split into two files: `src/PPO/ActorCritic.py` contains the network defined by Equations (4.3)–(4.6), and `src/PPO/ppo_agent.py` contains the training loop. The three public methods used by `main_train.py` are `select_action`, `evaluate_action` and `update`. Listing 5.1 shows the pseudocode of `select_action` and `evaluate_action`; Listing 5.2 shows the pseudocode of `update`.

```
def select_action(self, state, deterministic=False):
    # 1. Update running stats on the 15-dim vector block only.
    self._update_obs_stats(state)
    # 2. Normalise the vector block; leave the 720 LIDAR dims untouched.
    s = self._normalize_obs(state)
    with torch.no_grad():
        s_t = to_tensor(s, device=self.device).unsqueeze(0)
        if deterministic:
            mean = self.policy.actor_mean(self.policy.actor(s_t))
            action = torch.tanh(mean)
            return action.cpu().numpy().flatten(), 0.0, 0.0
        a, log_prob, _, value = self.policy.get_action_and_value(s_t)
    return a.cpu().numpy().flatten(), log_prob.item(), value.item()

def evaluate_action(self, state, action):
    # Called from main_train.py with action = info["executed_action"].
    # Returns the log-probability of the executed action under the current policy.
    s = self._normalize_obs(state)
    with torch.no_grad():
        s_t = to_tensor(s, device=self.device).unsqueeze(0)
```

```
        a_t = to_tensor(action, device=self.device).unsqueeze(0)
        _, log_prob, _, _ = self.policy.get_action_and_value(s_t, a_t)
    return log_prob.item()
```

Seudocódigo 5.1: Action sampling and evaluation, with support for the shield's credit-assignment correction.

5.4.1. Credit assignment under a shield

The interplay between the shield and the PPO update deserves a closer look, because it is the single most error-prone point of the framework. Let a_t be the action sampled from the policy and $a_t^\star = \Sigma(s_t, a_t)$ the action produced by the shield. The shield is a `gymnasium.Wrapper`, so the environment sees $a_t^\star$; the agent, however, only receives the value through the `executed_action` key of the `info` dictionary.

The training loop in `main_train.py` stores $a_t^\star$ (not a_t) in the rollout buffer and recomputes the log probability of $a_t^\star$ under the current policy via `evaluate_action`. Without this step, the gradient estimator would update the policy towards the action a_t that *was not executed*, which effectively decouples the policy from the environment and makes training degenerate. With this step, the policy learns to produce actions that are close to what the shield considered safe, which is precisely what is desired: the shield teaches the agent what *not* to do.

```
def update(self, memory):
    s, a, logp_old = load(memory, normalize_obs=self._normalize_obs)
    # 1. GAE backwards pass. Handles truncation via stored bootstrap values.
    with torch.no_grad():
        v_old = self.policy.get_value(s).squeeze(1)
    gae = 0.0; next_v = 0.0
    advantages = zeros_like(rewards)
    for t in reversed(range(len(rewards))):
        mask = 1.0 - dones[t]
        bootstrap = final_values[t] if truncated[t] else next_v
        delta = rewards[t] + gamma * bootstrap * mask - v_old[t]
        gae = delta + gamma * lam * mask * gae
        advantages[t] = gae
        next_v = v_old[t]
    returns = advantages + v_old
    advantages = (advantages - mean(advantages)) / (std(advantages) + 1e-7)
    # 2. Return normalisation by running std (no mean subtraction).
```

```
    self.ret_rms.update(returns); returns = returns / (std_running(self.ret_rms) + 1e-8)
    # 3. K epochs of PPO with optional KL early stop.
    kl_acc = 0.0
    for k in range(self.k_epochs):
        _, logp_new, entropy, v_new = self.policy.get_action_and_value(s, a)
        ratio = torch.exp(logp_new - logp_old)
        surr1 = ratio * advantages
        surr2 = torch.clamp(ratio, 1 - eps, 1 + eps) * advantages
        policy_loss = -torch.min(surr1, surr2).mean()
        if self.value_clip is not None:
            v_clip = v_old + torch.clamp(v_new - v_old, -xi, xi)
            value_loss = 0.5 * max((v_new - returns)**2, (v_clip - returns)**2).mean()
        else:
            value_loss = 0.5 * mse(v_new, returns)
        loss = policy_loss + c_v * value_loss - c_h * entropy.mean()
        self.optimizer.zero_grad(); loss.backward()
        clip_grad_norm(self.policy.parameters(), max_norm=1.0)
        self.optimizer.step()
        kl_acc += schulman_kl(logp_new, logp_old)
        if self.kl_target is not None and kl_acc / (k + 1) > self.kl_target:
            break
```

Seudocódigo 5.2: PPO update loop with GAE, value clipping, KL early stop and return normalisation.

The approximate KL in Listing 5.2 is the unbiased estimator of Equation (2.12); the threshold `kl_target` and the value-clip parameter ξ are exposed as command-line flags.

5.5. SHIELD IMPLEMENTATIONS

Both shields inherit from `gymnasium.Wrapper` and implement the same public interface (`reset`, `step`, `get_statistics`, `reset_statistics`). The interchangeability is what makes `-shield_type` a drop-in parameter of the training script.

5.5.1. Basic shield

The implementation of `CarlaSafetyShield` follows the correction law of Equations (4.17)–(4.18) with the angular sectors of Equations (4.12)–(4.14). The gains $k_{\mathrm{s}} = 2.5$ and $k_{\mathrm{b}} = 8.0$ and the thresholds ($\tau_{\mathrm{f}} = 0.15$, $\tau_{\mathrm{s}} = 0.04$, $\tau_{\ell} = 0.82$, $\tau_{h} = 0.60$) are defaults exposed as constructor arguments. The class keeps per-episode statistics of the sub-reason that triggered each intervention (`front`, `side_right`, `side_left`, `lane_right`, `lane_left`, `heading`), which are surfaced by `get_statistics()` and logged once per episode.

5.5.2. Adaptive-horizon shield

The adaptive shield is more involved. Listing 5.3 reproduces the structure of its `step` method.

```
def step(self, action):
    sem = self._analyze_semantic(self.last_obs, self.last_info)
    level, _ = self._get_risk_level_semantic(sem)
    H = self.HORIZON_CONFIG[level]["horizon"]
    self.stats[f"{level}_steps"] += 1

    carla_map = self._get_carla_map()
    ego = self._get_ego_vehicle()

    if self._check_trajectory_safety(action, H, level, carla_map, ego, sem):
        exec_action = action.copy(); shield_active = False
    else:
        exec_action = self._find_safe_action(H, level, carla_map, ego, sem)
        shield_active = True
        self._categorize_intervention(sem)
        self.shield_activations += 1

    obs, reward, done, truncated, info = self.env.step(exec_action)
    info.update({
        "shield_activated": shield_active, "risk_level": level,
        "executed_action": exec_action, "proposed_action": action,
        "horizon_used": H, "min_front_dist": sem["min_front_combined"],
        "min_front_dynamic": sem["min_front_dynamic"], ...
    })
    return obs, reward, done, truncated, info
```

Seudocódigo 5.3: Adaptive-horizon shield main loop. `sem` is the semantic analysis dictionary extracted from the `info` dictionary of the underlying environment.

The trajectory check is itself a loop over candidates that reuses `_check_trajectory_safety`. Listing 5.4 shows the essential structure: pedestrian emergency, LIDAR thresholds, bicycle-model prediction and a Waypoint-API check at every predicted position.

```
def _check_trajectory_safety(self, action, H, level, carla_map, ego, sem):
    k = self.HORIZON_CONFIG[level]["threshold_multiplier"]
    tau_f = self.front_threshold_base / k
    tau_s = self.side_threshold_base / k
    tau_l = max(self.lateral_threshold_base / k, 0.45)
    if self._is_lane_change_context():
        tau_l = max(tau_l, 1.2)

    if sem["nearest_pedestrian_m"] < self.PED_EMERGENCY_M:     # 4.0 m
        return False
    if sem["min_front_combined"] < tau_f: return False
    if sem["min_r_side_static"]  < tau_s: return False
    if sem["min_l_side_static"]  < tau_s: return False

    if ego is None or carla_map is None:
        return True    # Fallback: accept the action if CARLA objects unavailable.

    x, y, yaw = ego_state(ego); v = ego_speed(ego)
    traj = self.bicycle_model.predict_trajectory(x, y, yaw, v,
                                                 float(action[0]), float(action[1]), H)
    for (px, py, _) in traj[1:]:
        wp = carla_map.get_waypoint(carla.Location(px, py, 0.0),
                                    project_to_road=True,
                                    lane_type=carla.LaneType.Driving)
        if wp is None: return False
        lat_norm = lateral_offset_norm(px, py, wp)
        if lat_norm > tau_l: return False
    return True
```

Seudocódigo 5.4: Dual LIDAR/Waypoint trajectory check of the adaptive shield (simplified).

The candidate-search routine follows the priority-ordered policy described in Section 4.5.2. Each candidate is a hard-coded two-dimensional vector; the first that passes `_check_trajectory_safety` is returned, and if none passes, the fallback action of Equation (4.23) is returned.

5.5.3. Bicycle model

`BicycleModel.predict_trajectory` is a loop of H forward-Euler steps that implements Equations (2.21)–(2.24), with the arc-form integration of Equations (2.25)–(2.27) when $|\delta| \geq 10^{-4}$. The class is a plain Python object with no PyTorch or CARLA dependency; it is therefore trivially unit-testable, although the current test suite does not exercise it in isolation.

5.6. REWARD SHAPER

`CarlaRewardShaper` reads ~ 20 scalars from the `info` dictionary and produces the thirteen auxiliary quantities listed in Equations (4.25)–(4.34), plus the base reward, and returns the sum of Equation (4.24) as the shaped reward of the step. It also writes every sub-component back into the `info` dictionary under the keys `speed_bonus`, `lane_center_bonus`, `heading_bonus`, `smooth_penalty`, `invasion_penalty`, `road_penalty`, `progress_bonus`, `shield_intervention_pen`, `idle_penalty` and `drift_penalty`, so that the metrics logger can record the per-step reward decomposition. Three minor state variables are kept across steps:

- `_last_steering`: needed by the smoothness penalty of Equation (4.31).
- `_last_milestone`: needed by the progress bonus to avoid double-crediting the same $25\,\mathrm{m}$ segment.
- `_shield_grace_steps`: count-down of the grace period that temporarily disables the idle penalty after a shield activation.

The unit test `tests/test_reward_balance.py` feeds a crafted `info` dictionary to the shaper and asserts that each component of the shaped reward matches its analytical expression. The test runs in $< 1\,\mathrm{s}$ without a CARLA server and is therefore used as a pre-commit safeguard against regressions.

5.7. CURRICULUM MANAGER

`CurriculumManager` is a small self-contained class. Its only state is the tuple $(i, E_{\text{stage}}, C_{\text{rb}})$ where i is the current stage index. The main routine is `step(offroad_rate, crash_rate, avg_reward)`, which is called from `main_train.py` after each episode and returns a pair (target_npcs, event $\in$ {`none`, `advance`, `rollback`}). The training loop reacts to the event by updating `CarlaEnv.num_npc_vehicles` in place, so that no reconnection to CARLA is required (Section 4.7).

5.8. TRAINING AND EVALUATION PIPELINES

Both entry points (`main_train.py` and `main_eval.py`) share the function `build_env(args)` that constructs the wrapper chain of Figure 4.1 with the shield type, reward weights, curriculum NPC count and map chosen by the command line. Keeping the same builder on both sides guarantees that the evaluation environment matches the training environment exactly.

5.8.1. Training pipeline

Algorithm 1 summarises the training loop.

5.8.2. Evaluation pipeline

The evaluation pipeline (`main_eval.py`) is structurally simpler. It rebuilds the wrapper chain with the same `build_env` function, but sets all reward weights to zero in the shaper so that the reported episode reward is the pure CARLA base reward; the shield is still attached so that the action modifications are visible. A Matplotlib dashboard (`CarlaDashboard`) displays the LIDAR polar plot, the speed gauge, the lateral offset marker and a monospaced panel with live information (risk level, shield activation, steering, throttle/brake, total distance, collisions). At the end of the run a `SafetyMetricsReporter` prints a summary table that aggregates success rate, crash rate, off-road rate, shield activations and LIDAR-derived minimum distances.

Algorithm 1 High-level training loop (`main_train.py`).

Parse command-line arguments, create run directory, open SQLite metrics DB.
Instantiate CURRICULUMMANAGER and call BUILD_ENV with its initial stage.
Instantiate the PPOAGENT with dimensions read from the wrapper chain.
Initialise the rollout memory and the 100-episode sliding windows.
for `episode` $= 1, \ldots,$ `max_episodes` **do**
    $(\texttt{obs}, \_) \leftarrow$ `env.reset()`
    for `step` $= 1, \ldots,$ `max_steps` **do**
        $(a_t, \log p_t, V_t) \leftarrow$ `agent.select_action(obs)`
        $(\texttt{next\_obs}, r_t, \texttt{done}, \texttt{trunc}, \texttt{info}) \leftarrow$ `env.step`(a_t)
        $a_t^\star \leftarrow$ `info["executed_action"]`
        $\log p_t^\star \leftarrow$ `agent.evaluate_action`$(\texttt{obs}, a_t^\star)$
        $V_t^{\text{boot}} \leftarrow$ `agent.compute_bootstrap_value`$(\texttt{next\_obs})$ **if** `trunc and not done` **else** 0
        Append $(\texttt{obs}, a_t^\star, \log p_t^\star, r_t, \texttt{done}, \texttt{trunc}, V_t^{\text{boot}})$ to memory.
        if `timestep` mod `update_timestep` $= 0$ **then**
            `metrics` $\leftarrow$ `agent.update(memory)`
            Clear memory. Log `metrics`. Step the LR scheduler.
        end if
        if `done or trunc` **then break**
        end if
        `obs` $\leftarrow$ `next_obs`
    end for
    Update sliding windows; compute rates.
    $(\texttt{tgt\_npcs}, \texttt{event}) \leftarrow$ `curriculum.step(...)`
    if `event` $\neq$ `none` **then**
        `base_env.num_npc_vehicles` $\leftarrow$ `tgt_npcs`
    end if
    Log per-episode metrics. Save *best* checkpoint if $\overline{R}_{100}$ improved.
    if `episode` mod `ckpt_freq` $= 0$ **then** save periodic checkpoint.
    end if
end for
Save final checkpoint. Close environment and metrics DB.

5.9. PERSISTENCE AND REPRODUCIBILITY

5.9.1. Checkpoint format

Checkpoints are saved with `torch.save` as a dictionary of two entries: the `state_dict` of the `ActorCritic` network and the state of the `RunningMeanStd` normaliser. The second entry is essential for evaluation to produce meaningful observations—if it is missing, the running statistics would be rebuilt from scratch during evaluation and the input distribution would not match the one the policy was trained on. A shape check on load tolerates the legacy format (state dictionary only) for backwards compatibility with the earliest checkpoints.

5.9.2. Seeds and determinism

Two distinct seeds are used:

- `seed=42` for training. Propagated to NumPy, Python's `random` module, PyTorch, CARLA's Traffic Manager and the environment's episode-counter random state.
- `seed=100` for evaluation. Kept intentionally different from the training seed so that evaluation trajectories are not a subset of the trajectories the agent has already seen during learning.

Full bit-exact determinism across hardware is impossible because PyTorch's convolution and cuDNN implementations are not bit-reproducible and because CARLA's physics engine has a small floating-point divergence after thousands of ticks. However, determinism is sufficient for the first fifty episodes of a run, which is long enough to diagnose regressions introduced by a code change. The acceptance criterion AC-09 of Table 3.4 is formulated in exactly these terms.

5.9.3. Tooling

The source code is linted and formatted with `ruff`. Reward regressions are caught by `pytest tests/test_reward_balance.py`. The set of utility scripts under `utils/` provides helpers to list, inspect and compare checkpoints (`ModelManager`), to analyse the metrics of finished runs (`ExperimentAnalyzer`) and to generate configuration files from a template (`ConfigurationTemplate`). None of these utilities are required for training; they are purely auxiliary.

6. EXPERIMENTATION AND RESULTS

This chapter reports the empirical evaluation of the framework. It is organised around the four configurations that matter: the baseline agent with no shield (Section 6.3), the basic shield (Section 6.4), the adaptive-horizon shield (Section 6.5), and the effect of enabling or disabling the curriculum (Section 6.6). Section 6.1 fixes the experimental protocol and Section 6.2 formally defines every metric reported hereafter; Section 6.7 offers a cross-configuration discussion.

All numerical entries marked as *TBD* are to be filled in with the exact values obtained from the SQLite metrics database of each run. The SQL queries used to compute the reported statistics are documented in Appendix C.

6.1. EXPERIMENTAL PROTOCOL

6.1.0.0.1. Hardware and software.

All experiments are carried out on the workstation described in Section 3.3:

- CPU: *TBD*.
- GPU: *TBD*.
- System RAM: *TBD*.
- Operating system: Windows 11 Pro.
- Python: *TBD*.
- PyTorch: *TBD* with CUDA *TBD*.
- CARLA: 0.9.16, launched with `.\CarlaUE4.exe -RenderOffScreen -carla-port=2000`.

6.1.0.0.2. Benchmark scenario.

The main benchmark is the Town04 highway loop with clear weather (`ClearNoon`). The vehicle spawns at a random waypoint each episode and the goal is to travel at least $250\,\mathrm{m}$ along the map's route graph within 1000 simulator steps ($50\,\mathrm{s}$).

6.1.0.0.3. Training budget.

Every configuration is trained for $T = 2\,500$ episodes, with PPO updates every $2\,048$ steps ($\sim$ *TBD* updates per run).The learning rate starts at 10^{-4} and anneals cosine-style to 10^{-5}; the target KL for the early stop is 0.08; the entropy bonus is 0.02; the value-loss coefficient is 0.25; all other hyperparameters take the defaults of Appendix A.

6.1.0.0.4. Random seeds.

Every training run uses $\mathrm{seed} = 42$; every evaluation uses $\mathrm{seed} = 100$. Three independent runs per configuration are carried out when time permits, by varying the seed to $\{42, 43, 44\}$; otherwise the single $\mathrm{seed} = 42$ run is used. The reported aggregate statistics (mean, standard deviation, median) are computed across these three runs when applicable.

6.1.0.0.5. Curriculum.

Unless explicitly stated otherwise, the curriculum is enabled with $N_{\max} = 40$. The ablation of Section 6.6 disables it.

6.1.0.0.6. Evaluation protocol.

After training, each agent is evaluated on 10 episodes with $\mathrm{seed} = 100$. The shield configuration used at evaluation is identical to the one used during training: removing the shield at evaluation would compromise the comparability of the results because a policy trained with a shield has learned to rely on its interventions.

6.2. METRICS

The metrics reported throughout this chapter are exactly those produced by the training and evaluation pipelines. Their operational definitions are collected here to avoid ambiguity.

Reward/Raw_Episode. Undiscounted sum of shaped rewards of a single episode.

Reward/Average_100_Episodes. Rolling mean of *Reward/Raw_Episode* over the last 100 episodes.

Training/Success_Rate. Fraction of the last 100 episodes that ended with `info["arrive_dest"] = True`.

Training/Crash_Rate. Fraction of the last 100 episodes that ended with `info["collision"] = True`.

Training/Offroad_Rate. Fraction of the last 100 episodes that ended with `info["out_of_road"] = True`.

Safety/Shield_Activations. Number of steps in which the shield overrode the agent during the episode.

Safety/Shield_Rate. *Safety/Shield_Activations* divided by the number of steps in the episode.

Safety/Semantic/{Dynamic,Static,Pedestrian}_Interventions. Per-episode intervention count by threat category, as produced by `CarlaAdaptiveHorizonShield.get_statistics`.

Safety/Semantic/{Safe,Warning,Critical}_Step_Rate. Fractions of steps spent in each risk level.

CARLA/Mean_Speed_kmh. Arithmetic mean of `speed_kmh` across the steps of the episode.

CARLA/Speed_Compliance_Rate. Fraction of steps (with a positive speed limit) that satisfy $\texttt{speed\_kmh} \leq 1.05\,\texttt{speed\_limit\_kmh}$.

Safety/Min_{Vehicle,Pedestrian}_Distance_m. Minimum distance over the episode from the ego vehicle to the nearest vehicle / pedestrian.

Loss/Policy_Loss, Loss/Value_Loss, Loss/Grad_Norm, Training/Entropy, Training/Approx_KL, Training/Epochs_Run. Per-update diagnostics produced by the PPO loop.

6.3. BASELINE : SHIELD_TYPE=NONE

The baseline corresponds to an unconstrained PPO agent that follows exactly the same training protocol but without any shield. Its purpose is twofold: (i) it quantifies what a raw RL agent can learn on Town04 given the curriculum and the dense reward, and (ii) it provides the reference numbers against which every shielded configuration is compared.

6.3.0.0.1. Training curves.

Figure 6.1 shows the rolling reward, the success rate and the safety rates over the $2\,500$ training episodes. The agent reaches a final rolling reward of *TBD* with a success rate of *TBD* and a crash rate of *TBD*.

TBD: figure file, e.g. `figures/baseline_reward.pdf`

Figure 6.1.: Baseline run (`-shield_type none`). Top: rolling reward over 100 episodes. Bottom left: success, crash and off-road rates. Bottom right: number of shield activations per episode (identically zero for this configuration).

6.3.0.0.2. Curriculum trajectory.

Figure 6.2 shows the evolution of the curriculum stage. Rollbacks, if any, are visible as downward steps.

TBD: figure file, e.g. `figures/baseline_curriculum.pdf`

Figure 6.2.: Baseline run. Curriculum stage (0 to 4) across training episodes and associated NPC count.

6.3.0.0.3. Evaluation.

Table 6.1 summarises the 10 evaluation episodes.

Table 6.1.: Baseline evaluation, 10 episodes, $\mathrm{seed} = 100$.

Metric	**Baseline**
Average reward	TBD $\pm$ TBD
Success rate	TBD
Crash rate	TBD
Off-road rate	TBD
Timeout rate	TBD
Mean speed (km/h)	TBD
Speed-compliance rate	TBD

6.4. BASIC SHIELD

The second configuration enables the basic shield (`-shield_type basic`). Shield interventions are now logged per step, per episode and per semantic reason (`front`, `side_right`, `side_left`, `lane_right`, `lane_left`, `heading`).

6.4.0.0.1. Training curves.

Figure 6.3 mirrors Figure 6.1; the additional panel reports the shield-activation rate per episode. The final rolling reward is *TBD*; the final crash rate is *TBD*, compared to *TBD* for the baseline.

TBD: figure file, e.g. `figures/basic_training.pdf`

Figure 6.3.: Basic-shield run. Top: rolling reward. Bottom left: safety rates. Bottom right: shield-activation rate per episode.

6.4.0.0.2. Intervention breakdown.

Table 6.2 gives the mean number of interventions per episode by sub-reason, averaged over the last 100 training episodes.

Table 6.2.: Basic shield: interventions per episode by sub-reason (last 100 episodes).

Sub-reason	Mean interventions / episode
`front`	TBD
`side_right`	TBD
`side_left`	TBD
`lane_right`	TBD
`lane_left`	TBD
`heading`	TBD

6.4.0.0.3. Evaluation.

Table 6.3 reports the 10 evaluation episodes.

Table 6.3.: Basic shield evaluation, 10 episodes, $\mathrm{seed} = 100$.

Metric	Basic shield
Average reward	TBD $\pm$ TBD
Success rate	TBD
Crash rate	TBD
Off-road rate	TBD
Timeout rate	TBD
Mean speed (km/h)	TBD
Speed-compliance rate	TBD
Shield activations / ep	TBD

6.5. ADAPTIVE-HORIZON SHIELD

The third configuration enables the adaptive-horizon shield (`-shield_type adaptive`). In addition to the metrics reported for the basic shield, the adaptive shield exposes the time spent in each risk level and the breakdown of interventions by threat category (dynamic, static, pedestrian).

6.5.0.0.1. Training curves.

Figure 6.4 mirrors Figure 6.3 with an extra panel for the risk-level time fractions.

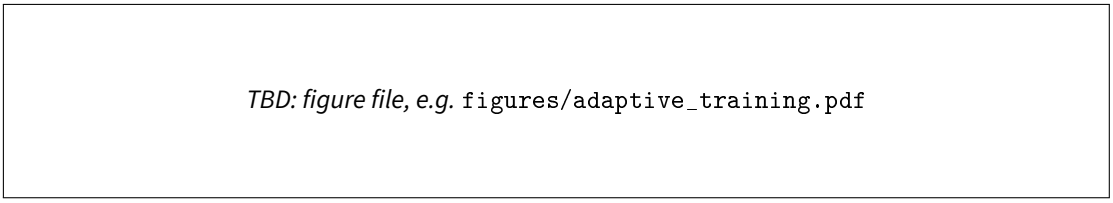

Figure 6.4.: Adaptive-shield run. Top-left: rolling reward. Top-right: safety rates. Bottom-left: shield-activation rate by threat category. Bottom-right: fraction of steps per risk level.

6.5.0.0.2. Risk distribution.

Table 6.4 reports the average time spent in each risk level over the last 100 training episodes.

Table 6.4.: Adaptive shield: average time per risk level (last 100 episodes).

Risk level	Fraction of steps
safe	TBD
warning	TBD
critical	TBD

6.5.0.0.3. Intervention breakdown.

Table 6.5 reports the intervention count per episode by threat category.

Table 6.5.: Adaptive shield: interventions per episode by threat category (last 100 episodes).

Category	Mean interventions / episode
dynamic	TBD
static	TBD
pedestrian	TBD

6.5.0.0.4. Evaluation.

Table 6.6 reports the 10 evaluation episodes.

Table 6.6.: Adaptive shield evaluation, 10 episodes, $\mathrm{seed} = 100$.

Metric	Adaptive shield
Average reward	TBD $\pm$ TBD
Success rate	TBD
Crash rate	TBD
Off-road rate	TBD
Timeout rate	TBD
Mean speed (km/h)	TBD
Speed-compliance rate	TBD
Shield activations / ep	TBD
Mean time in *critical* (%)	TBD

6.5.0.0.5. Cross-configuration comparison.

Table 6.7 aggregates the evaluation of the three configurations side-by-side.

Table 6.7.: Side-by-side evaluation comparison (10 episodes, seed 100).

Metric	None	Basic	Adaptive
Average reward	TBD	TBD	TBD
Success rate	TBD	TBD	TBD
Crash rate	TBD	TBD	TBD
Off-road rate	TBD	TBD	TBD
Speed-compliance rate	TBD	TBD	TBD
Min. vehicle distance (m)	TBD	TBD	TBD
Shield activations / ep	0	TBD	TBD

6.6. EFFECT OF THE CURRICULUM

To quantify the contribution of the curriculum we repeat the adaptive-shield configuration with `-no-curriculum`, i.e. with the manager disabled and the agent exposed to the maximum NPC count from episode 1. Everything else is held fixed.

6.6.0.0.1. Training curves.

Figure 6.5 compares the rolling reward of both runs. Without the curriculum, the agent takes *TBD* episodes to reach the reward level that the curriculum-enabled run reaches after *TBD* episodes; the final rolling crash rate without curriculum is *TBD* against *TBD* with curriculum.

TBD: figure file, e.g. `figures/curriculum_ablation.pdf`

Figure 6.5.: Curriculum ablation: adaptive shield with curriculum enabled (solid) vs. disabled (dashed). Top: rolling reward. Bottom: rolling crash rate.

6.6.0.0.2. Rollback analysis.

When the curriculum is enabled, Table 6.8 enumerates the stage transitions and rollbacks recorded during training.

Table 6.8.: Curriculum events during the adaptive-shield run.

Episode	Event	From stage	To stage
TBD	advance	TBD	TBD
TBD	advance	TBD	TBD
TBD	rollback	TBD	TBD
…	…	…	…

6.7. DISCUSSION

This section will discuss the trade-offs observed across configurations. The analysis will cover at least the following four axes; each is introduced here as a question that the empirical numbers of Sections 6.3–6.6 will answer.

1. **Safety vs. performance.** How much reward does a shield cost in exchange for a lower crash rate? Is the adaptive shield strictly Pareto-better than the basic shield, or does one of them dominate on one axis while the other dominates on the complementary axis?

2. **Intervention regime.** What fraction of a typical episode is spent with the shield active, and how does this fraction evolve during training? A decreasing shield-activation rate is consistent with the hypothesis that the policy is internalising the shield's behaviour.

3. **Threat composition.** Which fraction of interventions is due to dynamic threats (other vehicles), static threats (barriers, walls) or pedestrians? How do these fractions change across curriculum stages?

4. **Curriculum value.** Does the performance-driven curriculum accelerate learning and, more importantly,

does it reduce the worst-case transient crash rate of the adaptive-shield run? The empirical answer to this question informs the recommendation of whether to enable the curriculum by default.

Each enumerated item will be supported by a specific cross-reference to the tables and figures above. Once the TBD entries have been filled in, a concise summary (bulleted or tabular) will be inserted at the end of this section to provide a reader-friendly overview of the findings.

BIBLIOGRAFÍA

[1] World Health Organization, “Global status report on road safety 2023,” World Health Organization, Geneva, Switzerland, Tech. Rep., 2023.

[2] SAE International, “Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles,” SAE International, Warrendale, PA, USA, Standard J3016, 2021.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[4] B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. A. Al Sallab, S. Yogamani, and P. Pérez, “Deep reinforcement learning for autonomous driving: A survey,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 6, pp. 4909–4926, 2022.

[5] S. Aradi, “Survey of deep reinforcement learning for motion planning of autonomous vehicles,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 2, pp. 740–759, 2022.

[6] J. García and F. Fernández, “A comprehensive survey on safe reinforcement learning,” *Journal of Machine Learning Research*, vol. 16, no. 42, pp. 1437–1480, 2015.

[7] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, “Safe reinforcement learning via shielding,” in *Proc. 32nd AAAI Conf. Artificial Intelligence*, 2018, pp. 2669–2678.

[8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.

[9] A. Dosovitskiy, G. Ros, F. Codevilla, A. López, and V. Koltun, “CARLA: An open urban driving simulator,” in *Proc. 1st Conf. Robot Learning (CoRL)*, 2017, pp. 1–16.

[10] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. Boston, MA, USA: Springer, 2012.

[11] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. New York, NY, USA: John Wiley & Sons, 1994.

[12] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, “Policy gradient methods for reinforcement learning with function approximation,” in *Advances in Neural Information Processing Systems*, vol. 12, 1999, pp.

1057–1063.

[13] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, “Trust region policy optimization,” in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, 2015, pp. 1889–1897.

[14] M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Orsini, S. Girgin, R. Marinier, L. Hussenot, M. Geist, O. Pietquin, M. Michalski, S. Gelly, and O. Bachem, “What matters in on-policy reinforcement learning? a large-scale empirical study,” in *Int. Conf. Learning Representations (ICLR)*, 2021.

[15] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Mądry, “Implementation matters in deep policy gradients: A case study on PPO and TRPO,” in *Int. Conf. Learning Representations (ICLR)*, 2020.

[16] J. Schulman, “Approximating KL divergence,” http://joschu.net/blog/kl-approx.html, 2020, online technical note, accessed 2025.

[17] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *Int. Conf. Learning Representations (ICLR)*, 2016.

[18] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 22–31.

[19] N. Jansen, B. Könighofer, S. Junges, A. C. Serban, and R. Bloem, “Safe reinforcement learning using probabilistic shields,” in *Proc. 31st Int. Conf. Concurrency Theory (CONCUR)*, 2020, pp. 3:1–3:16.

[20] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, “Kinematic and dynamic vehicle models for autonomous driving control design,” in *Proc. IEEE Intelligent Vehicles Symposium (IV)*, 2015, pp. 1094–1099.

[21] J. Chen, B. Yuan, and M. Tomizuka, “Model-free deep reinforcement learning for urban autonomous driving,” in *Proc. 22nd IEEE Int. Conf. Intelligent Transportation Systems (ITSC)*, 2019, pp. 2765–2771.

[22] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger, “TransFuser: Imitation with transformer-based sensor fusion for autonomous driving,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 11, pp. 12 878–12 895, 2023.

[23] H. Krasowski, X. Wang, and M. Althoff, “Safe reinforcement learning for autonomous lane changing using set-based prediction,” in *Proc. 23rd IEEE Int. Conf. Intelligent Transportation Systems (ITSC)*, 2020, pp. 1–7.

[24] K. P. Wabersich and M. N. Zeilinger, “A predictive safety filter for learning-based control of constrained nonlinear dynamical systems,” *Automatica*, vol. 129, p. 109597, 2021.

[25] Q. Li, Z. Peng, L. Feng, Q. Zhang, Z. Xue, and B. Zhou, “MetaDrive: Composing diverse driving scenarios for generalizable reinforcement learning,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 3, pp. 3461–3475, 2023.

[26] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *Proc. 26th Int. Conf. Machine Learning (ICML)*, 2009, pp. 41–48.

[27] I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas, J. Schneider, N. Tezak, J. Tworek, P. Welinder, L. Weng, Q. Yuan, W. Zaremba, and L. Zhang, “Solving Rubik’s cube with a robot hand,” *arXiv preprint arXiv:1910.07113*, 2019.

[28] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018, pp. 1861–1870.

[29] B. P. Welford, “Note on a method for calculating corrected sums of squares and products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.

[30] SQLite Development Team, “Write-ahead logging (WAL),” https://www.sqlite.org/wal.html, 2024, sQLite technical documentation, accessed 2025.

A. HYPERPARAMETERS AND DEFAULT CONFIGURATION

This appendix collects the default values of every hyperparameter exposed by the command-line interfaces of `main_train.py` and `main_eval.py`, together with the constructor defaults of the reward shaper, the two shields and the curriculum manager.

A.1. PPO AGENT

Table A.1.: Default PPO hyperparameters.

Parameter	CLI flag / constructor	Default
Learning rate (initial)	`-lr`	1×10^{-4}
Learning rate (final)	internal	1×10^{-5}
LR schedule	internal	Cosine annealing
Discount factor γ	constructor	0.99
GAE λ	constructor	0.95
PPO clip ϵ	constructor	0.20
Value-loss coefficient	`-value_loss_coef`	0.25
Entropy coefficient	`-entropy_coef`	0.02
Rollout length	`-update_timestep`	$2\,048$
PPO epochs per update	`-k_epochs`	10
Approximate-KL target	`-kl_target`	0.08 (0 disables)
Max gradient ℓ_2-norm	constructor	1.0
Value clipping ξ	constructor	disabled
Hidden size	constructor	256
LIDAR encoder dims	constructor	$720 \to 256 \to 64$
$\log \sigma$ initial value	constructor	-0.70
$\log \sigma$ bounds	constructor	$[-3.0, -0.2]$
Obs. normaliser clip	`RunningMeanStd`	$[-5, 5]$
Obs. normalisation enabled	`-obs-norm`	enabled (vector block only)
Return normalisation	internal	enabled (std only)

A.2. ENVIRONMENT

A.3. REWARD SHAPER

A.4. SAFETY SHIELDS

A.5. CURRICULUM MANAGER

Table A.2.: Default environment hyperparameters.

Parameter	CLI flag / constructor	Default
CARLA host	`-host`	`localhost`
CARLA port	`-port`	2 000
Traffic Manager port	`-tm_port`	8 000
Map	`-map`	`Town04`
Number of NPC vehicles	`-num_npc`	40
Weather preset	`-weather`	`ClearNoon`
Target speed	`-target_speed_kmh`	50 km/h
Success distance	`-success_distance`	250 m
Synchronous mode	constructor	`True`
Simulator time step	constructor	0.05 s (20 Hz)
Number of LIDAR rays	constructor	240 (per channel)
LIDAR range	constructor	50 m
Max steps per episode	`-max_steps`	1 000
Training seed	`-seed`	42
Evaluation seed	constructor (eval)	100
Base success reward	constructor	+30
Base crash penalty	constructor	−10
Base off-road penalty	constructor	−20

Table A.3.: Default weights of the reward shaper. Symbols refer to the equations of Section 4.6.

Symbol	Description	CLI flag	Default
w_{v}	Speed-bonus scale	`-speed_weight`	0.08
w_{sm}	Smoothness penalty scale	`-smoothness_weight`	0.10
w_{lc}	Lane-centring scale	`-lane_centering_weight`	0.15
w_{h}	Heading-alignment scale	constructor	0.04
w_{inv}	Lane-invasion penalty	`-lane_invasion_penalty`	0.25
w_{edge}	Edge-warning scale	constructor	0.30
w_{prog}	Progress-milestone bonus	constructor	0.30
w_{wh}	Wrong-heading penalty	constructor	0.50
w_{road}	Off-road penalty (shaper)	`-off_road_penalty`	3.00
w_{sh}	Shield-intervention penalty	`-shield_intervention_penalty`	0.00
$w_{\mathrm{no-sh}}$	Shield-not-active bonus	constructor	0.02
w_{idle}	Idle penalty scale	`-idle_penalty_weight`	0.04
w_{dr}	Drift penalty scale	constructor	0.08
c_{alive}	Alive bonus	constructor	0.15
$v_{\min}$	Minimum moving speed	`-min_moving_speed_kmh`	5.0 km/h
v_g	Full speed-gate speed	constructor	10.0 km/h
c_{curv}	Curvature speed-target scale	constructor	0.4
η	Speed-limit tolerance	constructor	0.05
G	Shield grace-period length	constructor	10 steps
Δ_{prog}	Progress-milestone length	constant (`PROGRESS_MILESTONE_M`)	25 m

Table A.4.: Default hyperparameters of both shields. "Basic only" and "Adaptive only" columns indicate parameters that exist solely in one of the shields.

Parameter	**CLI flag**	**Scope**	**Default**
Front LIDAR threshold	`-front_threshold`	both	0.15
Side LIDAR threshold	`-side_threshold`	both	0.04
Lateral offset threshold	`-lateral_threshold`	both	0.65 (train), 0.82 (eval)
Heading threshold	constructor	basic only	0.60
Steering gain k_s	constructor	both	2.5
Brake gain k_b	constructor	both	8.0
Pedestrian emergency dist.	constant	adaptive only	$4.0\,\mathrm{m}$
Risk-level horizons (H)	internal	adaptive only	$1, 5, 10$
Threshold multipliers (κ)	internal	adaptive only	$1.0, 1.5, 2.0$
Frontal-risk boundaries	internal	adaptive only	$0.50,\ 0.20$
Lateral-risk boundaries	internal	adaptive only	$0.70,\ 0.85$
Lane-change lateral relaxation	internal	adaptive only	1.2
Minimum lateral floor	internal	adaptive only	0.45

Table A.5.: Default hyperparameters of the curriculum manager.

Parameter	**CLI flag / constructor**	**Default**
Curriculum enabled	`-curriculum` / `-no-curriculum`	enabled
Maximum NPC count	`-num_npc`	40
Number of stages	internal	5 (uniform in $[0, N_{\max}]$)
Min. episodes per stage	constructor	100
Rollback patience	constructor	50
Rollback counter decay	constructor	-1 per non-collapsing episode

B. OBSERVATION SPACE BREAKDOWN

This appendix describes every one of the 735 dimensions of the observation vector s_t. Indices are zero-based and match the slicing conventions used in `src/CARLA/Env/carla_env.py`.

B.1. LIDAR BLOCK (DIMENSIONS 0–719)

The first 720 dimensions contain three semantic LIDAR channels of 240 rays each. Each ray is normalised to $[0, 1]$, where 1 means a clear ray over the whole $50\,\mathrm{m}$ range and values close to 0 mean an obstacle within $\approx 1\,\mathrm{m}$. Ray $k \in \{0, \ldots, 239\}$ points towards the azimuth angle $\phi_k = k \cdot 1.5^\circ$ measured counter-clockwise from the vehicle's forward direction.

Table B.1.: LIDAR block of the observation vector.

Indices	Channel	Description
$[0, 240)$	Combined	All obstacles (vehicles, pedestrians, static, road edges).
$[240, 480)$	Dynamic	Only vehicles and pedestrians.
$[480, 720)$	Static	Only walls, barriers, poles and similar static objects.

B.2. LANE FEATURES (DIMENSIONS 720–727)

The next 8 dimensions are lane features computed from CARLA's Waypoint API at the vehicle's current position.

B.3. VEHICLE STATE (DIMENSIONS 728–729)

B.4. ROUTE INFORMATION (DIMENSIONS 730–734)

The final 5 dimensions provide look-ahead information along the route graph.

Table B.2.: Lane features.

Idx.	Name	Range	Meaning
720	`lateral_offset_norm`	$[-1, 1]$	Signed lateral offset of the vehicle with respect to the lane centre, normalised by the lane half-width.
721	`heading_error_norm`	$[-1, 1]$	Signed heading error between the vehicle and the lane tangent, normalised by $\pi/2$.
722	`on_edge_warning`	$[0, 1]$	Continuous warning that grows quadratically as the closest lane edge approaches.
723	`lane_width_norm`	$[0, 1]$	Lane width at the current position, normalised by a reference width (e.g. $4.0\,\mathrm{m}$).
724	`dist_left_edge_norm`	$[0, 1]$	Distance to the left lane edge, normalised by the lane width.
725	`dist_right_edge_norm`	$[0, 1]$	Distance to the right lane edge, normalised by the lane width.
726	`lane_change_permitted`	$\{0, 1\}$	1 if the Waypoint API flags either a left or a right lane change as permitted at the current position, 0 otherwise.
727	`road_curvature_norm`	$[-1, 1]$	Estimated signed road curvature, normalised by a reference curvature.

Table B.3.: Vehicle state features.

Idx.	Name	Range	Meaning
728	`speed_norm`	$[0, 1]$	Vehicle speed normalised by a reference speed (e.g. $150\,\mathrm{km/h}$).
729	`steering`	$[-1, 1]$	Last steering command actually applied to the vehicle (after shield correction).

B.5. SUMMARY

The full observation vector therefore has the structure

$$s_t = \underbrace{\left[\,\mathsf{LIDAR}_t^{\mathsf{comb}}\,\right]}_{240} \,\|\, \underbrace{\left[\,\mathsf{LIDAR}_t^{\mathsf{dyn}}\,\right]}_{240} \,\|\, \underbrace{\left[\,\mathsf{LIDAR}_t^{\mathsf{stat}}\,\right]}_{240} \,\|\, \underbrace{\left[\,\mathsf{Lane}_t\,\right]}_{8} \,\|\, \underbrace{\left[\,\mathsf{Veh}_t\,\right]}_{2} \,\|\, \underbrace{\left[\,\mathsf{Route}_t\,\right]}_{5} \in \mathbb{R}^{735}.$$

Only the rightmost 15 dimensions are normalised online by the running-mean estimator of Section 4.4.3.

Table B.4.: Route features.

Idx.	Name	Range	Meaning
730	`next_wp_angle_norm`	$[-1, 1]$	Signed angle to the next waypoint $5\,\mathrm{m}$ ahead, normalised by $\pi/2$.
731	`wp_angle_20m_norm`	$[-1, 1]$	Signed angle to the waypoint $20\,\mathrm{m}$ ahead (look-ahead for smooth turning).
732	`progress_norm`	$[0, 1]$	Fraction of the success distance traversed so far during the current episode.
733	`speed_limit_norm`	$[0, 1]$	Current speed limit normalised by the reference speed.
734	`speed_ratio`	$[0, 1]$	Ratio of the vehicle speed to the current speed limit.

C. REPRODUCIBILITY GUIDE

This appendix lists the exact commands needed to reproduce every experiment reported in Chapter 6, together with the SQL queries that were used to extract the aggregate statistics from the per-run SQLite databases.

C.1. ENVIRONMENT SETUP

1. Install the CARLA 0.9.16 binary distribution. On Windows, launch the server with

```
.\CARLA_0.9.16\CarlaUE4.exe -RenderOffScreen -carla-port=2000
```

2. Activate the project virtual environment (for example, the shared venv at `C:\Users\Isaac\Documents\TFG\Shielded-RL\TFG-RL`).

3. Verify that the Python dependencies are installed (PyTorch with CUDA support, Gymnasium, NumPy, Matplotlib, Streamlit, SQLite are part of the Python standard library).

C.2. TRAINING COMMANDS

```
# Baseline: PPO without any shield
python main_train.py --model_name baseline --shield_type none

# Basic shield
python main_train.py --model_name run_basic --shield_type basic

# Adaptive shield (recommended)
python main_train.py --model_name run_adaptive --shield_type adaptive

# Curriculum ablation (adaptive shield, curriculum disabled)
python main_train.py --model_name run_adaptive_nocur --shield_type adaptive --no-curriculum
```

Seudocódigo C.1: Commands used to reproduce the three training configurations of Chapter 6.

Each run writes its metrics to `runs/<model_name>_<shield>_<timestamp>/metrics.sqlite` and its

checkpoints to `data/models/<model_name>_<shield>_{best,final}.pth`.

C.3. EVALUATION COMMANDS

```
# Headless evaluation (metrics only), 10 episodes
python main_eval.py --model_name run_adaptive_adaptive_final.pth --shield_type adaptive \
                    --no_render --episodes 10

# Evaluation with the Matplotlib dashboard
python main_eval.py --model_name run_adaptive_adaptive_final.pth --shield_type adaptive \
                    --episodes 10
```

Seudocódigo C.2: Evaluation commands for the configurations above.

C.4. LIVE DASHBOARD

```
streamlit run dataVisualizer.py
```

Seudocódigo C.3: Launching the Streamlit dashboard.

The dashboard reads every `runs/*/metrics.sqlite` found under the project root and allows the user to select the run(s) to visualise. Because the database is opened in WAL mode, the dashboard can be launched while a training run is still in progress.

C.5. QUERY RECIPES

The following queries document how the empirical numbers that populate the tables of Chapter 6 are computed.

```
SELECT AVG(value) FROM metrics_episode
 WHERE key = 'Reward/Average_100_Episodes'
   AND step >= (SELECT MAX(step) - 99 FROM metrics_episode WHERE key = 'Reward/Average_100_

SELECT AVG(value) FROM metrics_episode
 WHERE key = 'Training/Crash_Rate'
```

```
    AND step >= (SELECT MAX(step) - 99 FROM metrics_episode WHERE key = 'Training/Crash_Rate

SELECT AVG(value) FROM metrics_episode
 WHERE key = 'Training/Offroad_Rate'
    AND step >= (SELECT MAX(step) - 99 FROM metrics_episode WHERE key = 'Training/Offroad_Ra
```

Seudocódigo C.4: Final 100-episode rolling reward and safety rates.

```
SELECT key, AVG(value) AS mean_fraction
  FROM metrics_episode
 WHERE key IN ('Safety/Semantic/Safe_Step_Rate',
               'Safety/Semantic/Warning_Step_Rate',
               'Safety/Semantic/Critical_Step_Rate')
 GROUP BY key;
```

Seudocódigo C.5: Fraction of time spent in each risk level (adaptive shield).

```
SELECT key, AVG(value) AS mean_count
  FROM metrics_episode
 WHERE key IN ('Safety/Semantic/Dynamic_Interventions',
               'Safety/Semantic/Static_Interventions',
               'Safety/Semantic/Pedestrian_Interventions')
 GROUP BY key;
```

Seudocódigo C.6: Average intervention count per episode by threat category.

```
-- Extract stage transitions by diffing the curriculum_NPC column between consecutive episo
WITH ordered AS (
  SELECT step, value,
         LAG(value) OVER (ORDER BY step) AS prev_value
    FROM metrics_episode
   WHERE key = 'Training/Curriculum_NPC'
)
SELECT step AS episode,
       CASE WHEN value > prev_value THEN 'advance'
            WHEN value < prev_value THEN 'rollback'
       END AS event,
```

```
        prev_value AS from_npcs,
        value      AS to_npcs
  FROM ordered
 WHERE value <> prev_value;
```

Seudocódigo C.7: Curriculum events during training.

C.6. CHECKPOINT INSPECTION

The utility class `ModelManager` in `utils/` exposes helpers to list, compare and inspect checkpoints. A quick sanity check can be performed with:

```
from utils import ModelManager
ModelManager.list_models()
ModelManager.print_model_info('./data/models/run_adaptive_adaptive_final.pth')
```

C.7. UNIT TESTS

The reward shaper tests run without a CARLA server:

```
python -m pytest tests/test_reward_balance.py -v
```

They exercise each component of the shaped reward of Equation (4.24) against a crafted `info` dictionary and ensure that disabling a weight zeroes out the corresponding component. They are run locally before every commit that touches `src/reward_shaper.py`.